TriaVet: Implementación en Prolog e Integración con Python de un Sistema Experto para el Triage de Urgencias en Perros y Gatos

Presentado por:

María José Ospina Polania

Arlen José Perdomo Rojas

Presentado a:

Juan José Ramos Diaz

Fundación Escuela Tecnológica – FET

Facultad (Ingeniería)

Programa (ingeniería de Software)

Neiva - Huila

2026

TriaVet: Implementación en Prolog e Integración con Python de un Sistema Experto para el Triage de Urgencias en Perros y Gatos

Este documento reúne los artefactos de entrega del sistema experto TriaVet: la ficha de definición del proyecto y de la pareja, el código fuente documentado, el diccionario de predicados y el script de pruebas por consola con su evidencia, documentada con capturas de pantalla tomadas en Visual Studio Code. También explica cómo la implementación cumple los requisitos técnicos de la base de conocimiento, el procesamiento de listas, los meta-predicados de recolección y severidad, y la integración entre Prolog y Python. Los archivos fuente acompañan este documento y se reproducen completos en los apéndices.

Ficha de Definición del Proyecto y Pareja

Integrantes de la Pareja

El proyecto se desarrolla en pareja. La Tabla 1 identifica a los integrantes y la asignación preliminar de roles. Ambos integrantes dominan Prolog y Python; el rol indica quién lidera cada frente de trabajo.

Tabla 1

Integrantes y Roles del Equipo

Integrante	Código	Correo institucional	Rol preliminar
[Nombre completo del integrante 1]	[Código]	[correo institucional]	Líder de Ingeniería del Conocimiento y Modelado Lógico (Prolog)
[Nombre completo del integrante 2]	[Código]	[correo institucional]	Líder de Arquitectura de Integración e Interfaz (Python)

Nota. Los datos entre corchetes deben ser completados por los integrantes.

Tema Seleccionado

Título del Sistema Experto

TriaVet: Sistema Experto Basado en Reglas para el Triage y la Orientación Diagnóstica Preliminar de Urgencias en Perros y Gatos.

Descripción del Problema

En Colombia, el 49 % de los hogares tiene perro y el 38 % tiene gato (Forbes Colombia, 2024). Además, la Ley 1774 de 2016 reconoce a los animales como seres sintientes, lo que refuerza la responsabilidad de sus tutores de procurarles atención oportuna. Cuando una mascota enferma, el tutor debe decidir si esperar, programar una cita o acudir a urgencias. Toma esa decisión sin formación clínica, ante un paciente que no puede describir lo que siente y con signos que suelen ser inespecíficos: un vómito puede deberse a una indiscreción alimentaria, pero también a una intoxicación, a una obstrucción intestinal o a una parvovirosis.

Decidir de forma intuitiva conduce a dos errores. El primero es subestimar un cuadro crítico, como la dilatación-torsión gástrica, la obstrucción uretral felina o el golpe de calor, y perder horas decisivas. El segundo es sobreestimar un cuadro leve y acudir a urgencias, con un costo mayor y congestión del servicio. Ruys et al. (2012) mostraron que una lista de triage veterinario estructurada clasifica la urgencia mucho mejor que el criterio intuitivo: en los casos más críticos coincidió con un equipo experto en el 95,5 % de los pacientes, frente al 44,8 % del triage intuitivo.

El usuario principal del sistema es el tutor de perros y gatos. Los usuarios secundarios son el personal auxiliar de clínicas veterinarias, que puede usarlo para el pretriage; los refugios y fundaciones, y los estudiantes de medicina veterinaria. Los médicos veterinarios no son usuarios finales, sino la fuente y los validadores del conocimiento.

Justificación como Sistema Basado en Reglas

El conocimiento que aplica un veterinario al evaluar una urgencia es heurístico y se expresa de forma natural como reglas del tipo SI-ENTONCES (Giarratano y Riley, 2005). Las entradas del tutor rara vez conducen directamente a una conclusión: primero deben inferirse estados intermedios, como la deshidratación, el abdomen agudo, los signos de shock o la retención urinaria, y a partir de ellos, junto con la especie, la edad y los antecedentes, se plantean los diagnósticos. Además, un mismo signo es compatible con varias enfermedades, por lo que el sistema debe sostener varias hipótesis con un grado de certeza (Shortliffe y Buchanan, 1975) y descartar las que otra hipótesis explica mejor.

La urgencia tampoco depende solo del diagnóstico: un signo de alarma eleva la prioridad al máximo y la vulnerabilidad del paciente la agrava. Un formulario condicional tendría que anticipar con estructuras if–else cada combinación posible de respuestas, no podría manejar información incompleta y no podría explicar su resultado. Por estas razones el problema requiere un motor de inferencia lógica, que Prolog ofrece de forma nativa mediante la unificación, el encadenamiento hacia atrás y el retroceso (Bratko, 2012).

Diagnósticos y Resultados Esperados

La Tabla 2 presenta el catálogo de diagnósticos que el sistema puede concluir. Para cada caso, el sistema entrega además la prioridad global, una recomendación de cuándo buscar atención, una indicación de qué hacer mientras tanto, el porcentaje de certeza y los hallazgos que sustentan cada diagnóstico.

Tabla 2

Catálogo de Diagnósticos de TriaVet

Identificador en Prolog	Diagnóstico	Especie	Severidad base
`parvovirosis`	Parvovirosis canina	Perro	Alto
`moquillo`	Moquillo canino (distemper)	Perro	Alto
`dilatacion_torsion_gastrica`	Dilatación-torsión gástrica	Perro	Crítico
`obstruccion_uretral`	Obstrucción uretral felina	Gato macho	Crítico
`cistitis`	Cistitis / infección urinaria baja	Perro y gato	Medio
`intoxicacion`	Intoxicación	Perro y gato	Según la sustancia
`golpe_de_calor`	Golpe de calor	Perro y gato	Crítico
`obstruccion_intestinal`	Obstrucción intestinal por cuerpo extraño	Perro y gato	Alto
`enfermedad_renal_cronica`	Enfermedad renal crónica	Perro y gato	Medio
`gastroenteritis_aguda`	Gastroenteritis aguda inespecífica	Perro y gato	Bajo
`sin_diagnostico_concluyente`	Valor por defecto cuando ningún perfil se cumple	—	Bajo

Nota. La severidad base puede subir por signos de alarma o por factores agravantes. Elaboración propia.

Implementación de los Requisitos Técnicos

Base de Conocimiento y Encadenamiento Lógico

Hechos Estructurados

El archivo `base_conocimiento.pl` contiene 224 hechos de dominio distribuidos en 23 predicados, todos de aridad igual o mayor que dos. Los hechos de cada predicado están agrupados de forma contigua y organizados en trece bloques comentados de la Sección 1 del archivo. La Tabla 3 resume su contenido.

Tabla 3

Hechos de la Base de Conocimiento por Predicado

Predicado	Hechos	Contenido
`signo/2`	37	Signo clínico y sistema orgánico al que pertenece.
`antecedente/2`	16	Vacunación, condición del paciente, raza, exposiciones e ingestas.
`especie_afectada/2`	16	Especies en las que aplica cada diagnóstico.
`toxicidad/3`	13	Nivel de toxicidad de cada sustancia según la especie.
`nombre_diagnostico/2`, `curso/2`, `requisito/2`, `apoyo/2`, `minimo_apoyo/2`, `certeza_base/2`, `severidad_base/2`	77	Perfil completo de los diez diagnósticos y del valor por defecto (11 hechos por predicado).
`indicacion/2`	12	Qué hacer mientras se consigue atención veterinaria.
`nombre_grupo/2`	9	Nombres legibles de los grupos de hallazgos.
`sustancia/2`	8	Relación entre la ingesta reportada y la sustancia tóxica.
`indicadores/2`	6	Listas de hallazgos que definen los estados intermedios.
`recomendacion/2`	6	Recomendación para cada nivel de prioridad.
`excluye/2`	5	Diagnóstico dominante y diagnóstico que excluye.
`umbral_edad/3`, `nivel_valor/2`, `siguiente_nivel/2`	12	Etapas de vida por especie y escala de severidad.
`signo_alarma/2`, `especie_paciente/2`, `sexo_paciente/2`	7	Signos de alarma y valores admitidos para el paciente.

Nota. Total: 224 hechos. Elaboración propia.

Reglas de Inferencia con Encadenamiento

La base contiene 98 cláusulas de reglas en 55 predicados, organizadas en los tres niveles que muestra la Tabla 4. Las reglas de cada nivel consultan reglas del nivel inferior, de modo que una conclusión de Nivel 3 nunca se obtiene consultando solo hechos base.

Tabla 4

Organización de las Reglas por Nivel de Inferencia

Nivel	Predicados principales	Función
Nivel 1: validación preliminar	`hallazgo_valido/1`, `normalizar_entrada/3`, `datos_paciente_validos/1`, `etapa_vida/3`, `etapa/2`	Clasifican los datos de entrada y verifican condiciones elementales.

Nivel	Predicados principales	Función
Nivel 2: inferencia intermedia	`cuadro_digestivo/1`, `deshidratacion/1`, `signos_de_shock/1`, `signo_de_alarma/1`, `abdomen_agudo/1`, `retencion_urinaria/1`, `hipertermia_probable/1`, `sin_proteccion_vacunal/1`, `paciente_vulnerable/1`, `cumple_perfil/2`	Combinan reglas de Nivel 1 y hechos para deducir estados que el usuario no ingresa directamente.
Nivel 3: conclusión final	`candidato/2` (una cláusula por diagnóstico), `excluido/2`, `diagnostico_posible/2`	Integran los estados intermedios para concluir los diagnósticos y aplicar los descartes.

Nota. Además de los tres niveles, la Sección 2 contiene las utilidades de listas y las Secciones 6 y 7 el cálculo de severidad, la recolección y los predicados públicos. Elaboración propia.

La Figura 1 muestra la cadena completa para la dilatación-torsión gástrica: la regla de Nivel 3 depende del estado intermedio `abdomen_agudo/1` y de `cumple_perfil/2`, que a su vez usan las verificaciones de listas de Nivel 1.

Figura 1

Encadenamiento de Tres Niveles para la Dilatación-Torsión Gástrica

```
% ---------- Nivel 3: conclusión ----------
% Dilatación-torsión gástrica: abdomen agudo con distensión y arcadas.
candidato(Caso, dilatacion_torsion_gastrica) :-
    abdomen_agudo(Caso),
    cumple_perfil(Caso, dilatacion_torsion_gastrica).

% ---------- Nivel 2: estados intermedios ----------
abdomen_agudo(caso(_, _, _, H)) :-
    alguno_presente([dolor_abdominal, abdomen_distendido], H),
    alguno_presente([vomito_persistente, arcadas_improductivas], H).

cumple_perfil(caso(Especie, _, _, H), Diagnostico) :-
    especie_afectada(Diagnostico, Especie),
    requisito(Diagnostico, Requisitos),
    contiene_todos(Requisitos, H),
    apoyo(Diagnostico, Apoyo),
    contar_presentes(Apoyo, H, N),
    minimo_apoyo(Diagnostico, Minimo),
    N >= Minimo.

% ---------- Nivel 1: verificación de listas ----------
contiene_todos([], _).
contiene_todos([R|Rs], Hallazgos) :-
    presente(R, Hallazgos),
    contiene_todos(Rs, Hallazgos).
```

Nota. Fragmento literal de `base_conocimiento.pl`. Elaboración propia.

Prevención de Ciclos

El grafo de dependencias es acíclico y está documentado en la cabecera del archivo: cada regla consulta solo reglas de su mismo nivel o de uno inferior, y ninguna se invoca a sí misma como primera meta. El caso delicado es la exclusión entre diagnósticos: `excluido/2` consulta `candidato/2` y no `diagnostico_posible/2`, por lo que el sistema no entraría en un bucle aunque existieran exclusiones mutuas. Los predicados recursivos sobre listas consumen la cabeza de la lista antes de la llamada recursiva, de modo que siempre terminan. Las metas de cada regla se ordenan de manera que las variables estén instanciadas antes de las comparaciones y de las negaciones.

Procesamiento de Listas y Control de Flujo Lógico

Uso de Listas

Python envía a Prolog la lista de hallazgos observados. `normalizar_entrada/3` la ordena y elimina duplicados con `sort/2`, y `particionar/3` separa los hallazgos reconocidos de los que no existen en la base, que se devuelven como ignorados. Sobre la lista válida trabajan `contiene_todos/2`, que verifica que los requisitos de un diagnóstico sean un subconjunto de los hallazgos; `contar_presentes/3`, que cuenta cuántos signos de apoyo se cumplen, e `interseccion/3`, que filtra los hallazgos que sustentan cada conclusión. La Figura 2 muestra dos de estos predicados.

Figura 2

Predicados Recursivos de Procesamiento de Listas

```
%!  contar_presentes(+Lista, +Hallazgos, -N) is det.
%   Cuenta cuántos elementos de Lista aparecen en Hallazgos.
%   CORTE: la segunda y la tercera cláusula son mutuamente excluyentes
%   (el elemento está o no está); el corte impide que, al retroceder, la
%   tercera cláusula vuelva a contar un elemento que sí estaba presente.
contar_presentes([], _, 0).
contar_presentes([X|Xs], Hallazgos, N) :-
    presente(X, Hallazgos), !,
    contar_presentes(Xs, Hallazgos, N0),
    N is N0 + 1.
contar_presentes([_|Xs], Hallazgos, N) :-
    contar_presentes(Xs, Hallazgos, N).


%!  particionar(+Lista, -Validos, -Invalidos) is det.
%   Separa los hallazgos reconocidos por la base de conocimiento de los
%   que no lo son (errores de digitación, signos fuera del dominio).
%   CORTE: un elemento es válido o inválido, nunca ambas cosas.
particionar([], [], []).
particionar([H|T], [H|Validos], Invalidos) :-
    hallazgo_valido(H), !,
    particionar(T, Validos, Invalidos).
particionar([H|T], Validos, [H|Invalidos]) :-
    particionar(T, Validos, Invalidos).
```

Nota. Fragmento literal de `base_conocimiento.pl`. Elaboración propia.

Operador de Corte y Negación por Fallo

La Tabla 5 resume dónde se usan el corte y la negación por fallo y por qué. Cada uso está justificado además en el comentario del predicado correspondiente. En las reglas con salida, como `etapa_vida/3`, la variable de salida se unifica después del corte; así la regla también responde correctamente cuando se consulta con la salida ya instanciada, por ejemplo `etapa_vida(perro, 5, adulto)`, que debe fallar.

Tabla 5

Uso del Corte y de la Negación por Fallo

Mecanismo	Predicados	Justificación
Corte entre cláusulas excluyentes	`contar_presentes/3`, `interseccion/3`, `particionar/3`, `etapa_vida/3`, `severidad/3`, `subir_niveles/3`, `prioridad_caso/3`, `mayor_nivel/3`	Impide que el retroceso pruebe una cláusula alternativa que produciría un resultado erróneo o repetido.

Mecanismo	Predicados	Justificación
Corte con valor por defecto	`etapa_vida/3` (adulto), `toxicidad_para/3`, `agravantes/3`, `conclusiones/2`, `indicacion_para/3`, `diagnosticar_todos/5`, `analizar_caso/10`	Garantiza una respuesta cuando ninguna condición específica se cumple; Python nunca recibe un fallo.
Corte de "basta uno"	`alguno_presente/2`, `signo_de_alarma/1`, `excluido/2`, `paciente_vulnerable/1`	Detiene la búsqueda al encontrar la primera evidencia suficiente.
Negación por fallo	`sin_proteccion_vacunal/1`, `candidato/2` (enfermedad renal crónica y gastroenteritis), `diagnostico_posible/2`	Descarta un diagnóstico si hay vacunación completa, si el paciente es cachorro, si hay signos de alarma o si otro diagnóstico lo excluye.

Nota. En todas las negaciones el caso, y el diagnóstico cuando aplica, ya están instanciados antes de evaluar `\+`. Elaboración propia.

La Figura 3 muestra la negación por fallo en dos niveles: el descarte por vacunación completa y la exclusión entre diagnósticos, en la que `Diagnostico` ya está instanciado por `candidato/2` cuando se evalúa `\+ excluido/2`.

Figura 3

Negación por Fallo y Exclusión entre Diagnósticos

```
%!  sin_proteccion_vacunal(+Caso) is semidet.
%   NEGACIÓN POR FALLO: el paciente se considera sin protección si NO se
%   reporta vacunación completa. H está instanciada (viene del caso).
sin_proteccion_vacunal(caso(_, _, _, H)) :-
    \+ presente(vacunacion_completa, H).


%!  excluido(+Caso, +Diagnostico) is semidet.
%   Un diagnóstico se excluye si otro diagnóstico dominante es candidato.
%   CORTE: basta con encontrar una exclusión.
%   Llama a candidato/2 y no a diagnostico_posible/2: así se evitan ciclos.
excluido(Caso, Diagnostico) :-
    excluye(Dominante, Diagnostico),
    candidato(Caso, Dominante), !.


%!  diagnostico_posible(+Caso, ?Diagnostico) is nondet.
%   NEGACIÓN POR FALLO: Diagnostico ya está instanciado por candidato/2
%   cuando se evalúa \+ excluido/2.
diagnostico_posible(Caso, Diagnostico) :-
    candidato(Caso, Diagnostico),
    \+ excluido(Caso, Diagnostico).
```

Nota. Fragmento literal de `base_conocimiento.pl`. Elaboración propia.

Meta-predicados de Recolección y Severidad

El predicado `conclusiones/2` usa `setof/3` para reunir en una sola lista, ordenada y sin duplicados, todos los diagnósticos posibles de un caso. Como `setof/3` falla cuando no hay soluciones, una cláusula posterior al corte devuelve el valor por defecto `[sin_diagnostico_concluyente]`. Sobre este predicado se construyen `diagnosticar_todos/5`, el predicado principal de recolección, y `analizar_caso/10`, que usa `findall/3` para detallar cada conclusión. `catalogo_hallazgos/2` emplea `setof/3` agrupando por grupo para generar el menú de la consola. La Figura 4 muestra la recolección y el cálculo de severidad.

Figura 4

Recolección con setof/3 y Cálculo de Severidad

```
%!  conclusiones(+Caso, -Diagnosticos) is det.
%   RECOLECCIÓN con setof/3: agrupa en una sola lista ordenada y SIN
%   DUPLICADOS todos los diagnósticos posibles del caso.
%   CORTE + FALLBACK:
%     - sin hallazgos válidos no se evalúa nada (lista vacía);
%     - setof/3 falla si no hay soluciones; en ese caso la última cláusula
%       devuelve el valor por defecto [sin_diagnostico_concluyente].
conclusiones(caso(_, _, _, []), []) :- !.
conclusiones(Caso, Diagnosticos) :-
    setof(D, diagnostico_posible(Caso, D), Diagnosticos), !.
conclusiones(_, [sin_diagnostico_concluyente]).

%!  severidad(+Caso, +Diagnostico, -Nivel) is det.
%   1) Un signo de alarma fija el nivel en critico.
%   2) En otro caso: nivel base del diagnóstico (o de la sustancia tóxica),
%      subido un nivel por cada factor agravante.
%   CORTE: la primera cláusula, cuando aplica, anula la segunda.
severidad(Caso, _, Nivel) :-
    signo_de_alarma(Caso), !,
    Nivel = critico.
severidad(Caso, Diagnostico, Nivel) :-
    nivel_base(Caso, Diagnostico, Base),
    agravantes(Caso, Diagnostico, Agravantes),
    length(Agravantes, K),
    subir_niveles(Base, K, Nivel).
```

Nota. Fragmento literal de `base_conocimiento.pl`. Elaboración propia.

Cada diagnóstico recibe un nivel de la escala de la Tabla 6, con un peso numérico que se usa para ordenar los resultados y calcular la prioridad del caso.

Tabla 6

Escala de Severidad y Prioridad

Nivel	Peso	Recomendación
Crítico	4	Urgencia vital: acudir de inmediato a un servicio veterinario de urgencias.
Alto	3	Atención veterinaria el mismo día, en las próximas horas.
Medio	2	Consulta veterinaria en las próximas 24 a 48 horas, vigilando la evolución.
Bajo	1	Vigilar en casa y consultar si los signos persisten o empeoran.
Indeterminado	—	No se ingresaron hallazgos válidos.
Datos no válidos	—	La especie, la edad o el sexo están fuera del dominio.

Nota. Elaboración propia.

Los criterios se aplican en este orden de precedencia: (a) un signo de alarma, como la dificultad respiratoria, las convulsiones, el colapso o los signos de shock, fija el nivel en crítico sin importar el diagnóstico; (b) en otro caso se toma el nivel base del diagnóstico, que en las intoxicaciones depende de la sustancia y la especie; (c) cada factor agravante de un cuadro agudo, es decir, paciente vulnerable, deshidratación o compromiso neurológico, sube un nivel, con crítico como tope; y (d) la prioridad del caso es el nivel más alto entre sus diagnósticos. El sistema calcula además un factor de certeza simplificado: la certeza base del diagnóstico más 0,10 por cada signo de apoyo presente, con un tope de 0,95.

Integración Prolog y Python

El módulo `motor.py` define la clase `MotorTriaVet`, que carga `base_conocimiento.pl` mediante la biblioteca PySwip (Tekol, s. f.) y se comunica directamente con el motor de SWI-Prolog (Wielemaker et al., 2012). Cada método arma la consulta, la ejecuta y convierte la respuesta en objetos de Python. Los valores de entrada viajan como átomos entre comillas simples, de modo que un texto como *Fiebre* no se interprete como una variable de Prolog. La Figura 5 muestra el método que invoca el predicado principal.

Figura 5

Invocación de analizar_caso/10 desde Python

```
def analizar_caso(self, especie, edad, sexo, hallazgos: Iterable[object]) -> ResultadoCaso:
    """Invoca analizar_caso/10 y devuelve la respuesta completa."""
    consulta = self.consulta_analisis(especie, edad, sexo, list(hallazgos))
    soluciones = self.consultar(consulta, max_resultados=1)
    if not soluciones:
        # analizar_caso/10 tiene una cláusula por defecto, así que esto
        # solo ocurriría si la base de conocimiento estuviera incompleta.
        raise ErrorMotor("Prolog no devolvió respuesta para el caso.")
    s = soluciones[0]
    return ResultadoCaso(
        consulta=consulta,
        validos=_lista_textos(s["Validos"]),
        ignorados=_lista_textos(s["Ignorados"]),
        alarmas=_lista_textos(s["Alarmas"]),
        diagnosticos=[_a_diagnostico(f) for f in s["Diagnosticos"]],
        prioridad=_texto(s["Prioridad"]),
        recomendacion=_texto(s["Recomendacion"]),
    )
```

Nota. Fragmento literal de `motor.py`. Elaboración propia.

Python no contiene lógica de inferencia: no valida hallazgos, no descarta diagnósticos ni calcula severidades. Esas tareas residen en Prolog, que además devuelve los resultados ya ordenados. La consola interactiva `main.py` solo lee del teclado los datos del paciente y los hallazgos, que pueden escribirse por número del menú o por nombre, y muestra la respuesta; toda la interacción ocurre por consola, sin interfaz gráfica.

Código Fuente Documentado

La Tabla 7 describe los archivos que componen el código fuente. Todos están comentados, y los predicados de Prolog siguen la notación PlDoc, que documenta el modo de instanciación de cada argumento. Los Apéndices A a D reproducen los archivos completos.

Tabla 7

Archivos del Código Fuente

Archivo	Propósito	Contenido principal
`base_conocimiento.pl`	Base de conocimiento y motor de inferencia.	Siete secciones comentadas: hechos, utilidades de listas, reglas de los Niveles 1, 2 y 3, severidad y prioridad, y recolección y predicados públicos (Apéndice A).

Archivo	Propósito	Contenido principal
`motor.py`	Puente entre Python y SWI-Prolog con PySwip.	Clase `MotorTriaVet`, conversión de valores entre Python y Prolog, y formato de la respuesta (Apéndice B).
`main.py`	Consola interactiva.	Menú numerado de hallazgos, captura de datos y presentación del resultado (Apéndice C).
`pruebas.py`	Script de pruebas por consola.	Diez casos de prueba documentados y escritura de la evidencia (Apéndice D).
`evidencia/`	Evidencia de ejecución.	`salida_pruebas.txt` con la salida completa (Apéndice E), `prueba_<n>.txt` con la salida de cada caso suelto y `capturas/` con las capturas de este documento.
`.vscode/tasks.json`	Tareas de Visual Studio Code.	Una tarea por cada caso obligatorio, una para todas las pruebas y otra para la consola interactiva (Terminal > Run Task).
`docs/`, `README.md`	Documentación.	Ficha del proyecto, diccionario de predicados e instrucciones de ejecución.

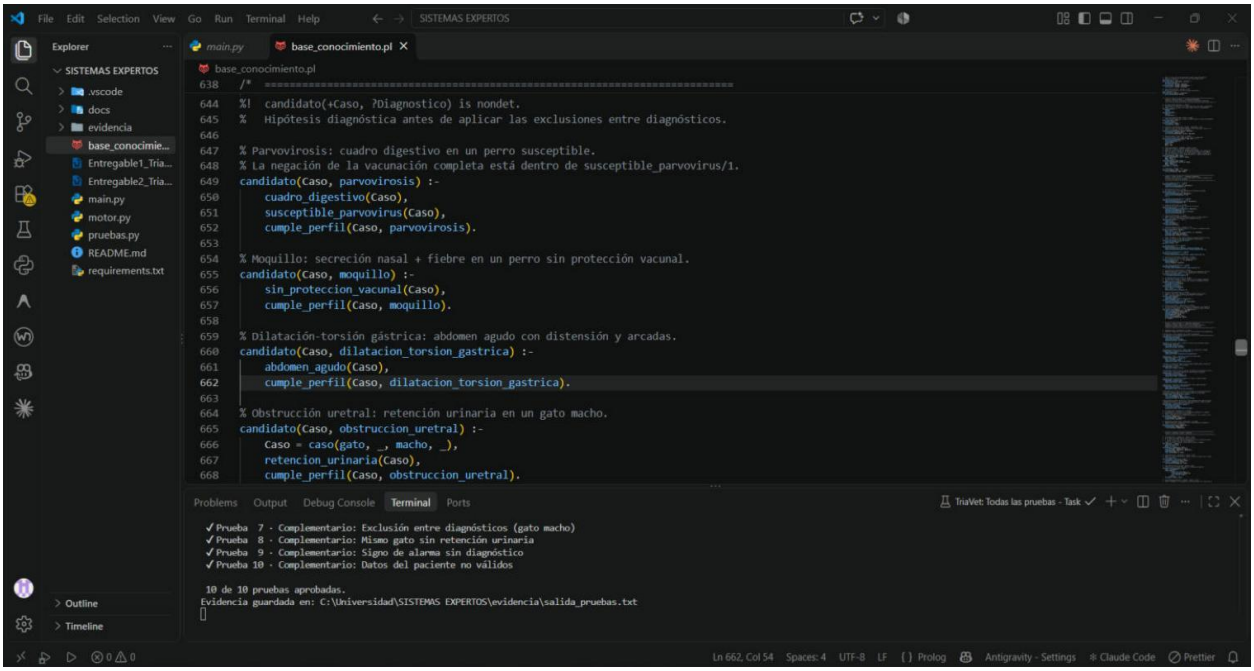
Nota. Requisitos: SWI-Prolog 9.x o 10.x, Python 3.10 o superior y PySwip 0.3.3 o superior. Elaboración propia.

Para ejecutar la consola interactiva se usa el comando `python main.py`, y para las pruebas, `python pruebas.py`; con un número, por ejemplo `python pruebas.py 3`, se ejecuta solo ese caso. La base de conocimiento también puede consultarse sin Python, con `swipl base_conocimiento.pl`.

El proyecto se desarrolló y ejecutó en Visual Studio Code. La Figura 6 muestra la base de conocimiento abierta en el editor, en las reglas de Nivel 3, con la terminal integrada debajo. La Figura 7 muestra el módulo `motor.py`, donde se construyen las consultas que se envían a Prolog.

Figura 6

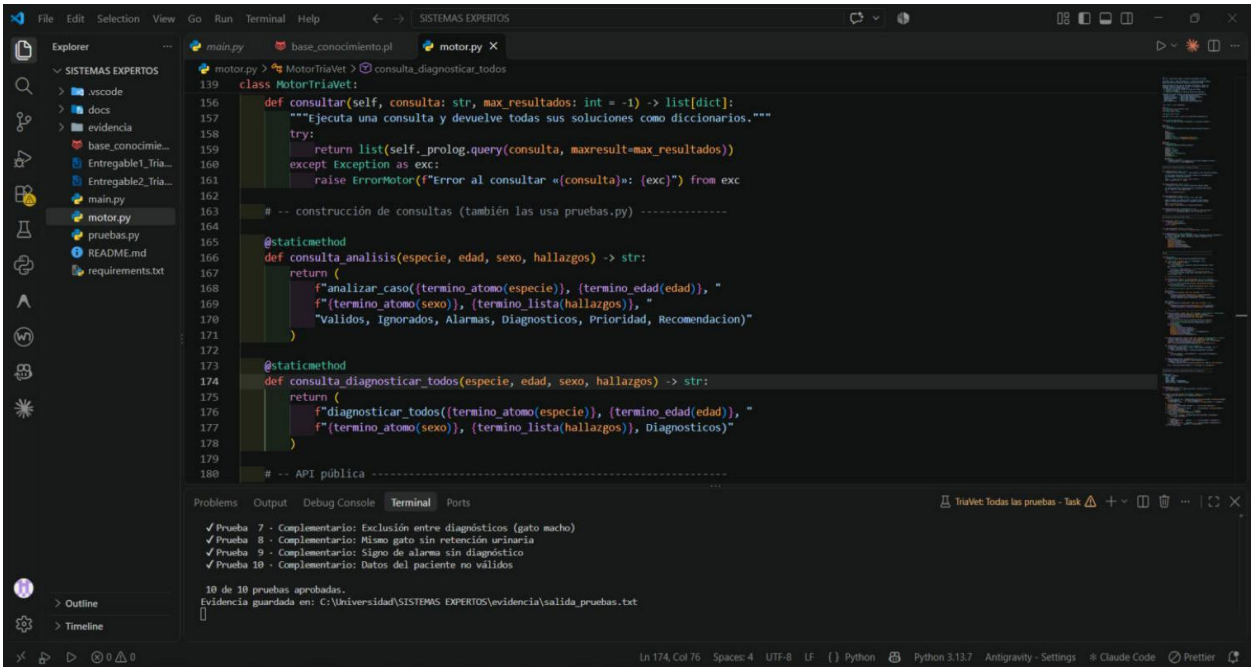
Base de Conocimiento Abierta en Visual Studio Code



Nota. Captura de pantalla de Visual Studio Code con las reglas `candidato/2` de `base_conocimiento.pl` y, debajo, el final de la ejecución de las pruebas.

Figura 7

Módulo motor.py Abierto en Visual Studio Code



Nota. Captura de pantalla de Visual Studio Code con los métodos de `MotorTriaVet` que construyen las consultas a Prolog.

Diccionario de Predicados

En la notación de modos, `+Arg` indica un argumento de entrada que debe llegar instanciado, enviado desde Python o desde la regla que llama; `-Arg`, un argumento de salida que el predicado instancia y que se devuelve a Python, y `?Arg`, un argumento que funciona instanciado o libre. Internamente, un caso se representa como `caso(Especie, EdadMeses, Sexo, Hallazgos)`. La Tabla 8 documenta los predicados públicos que invoca Python y la Tabla 9, los principales predicados internos.

Tabla 8

Predicados Públicos Invocados desde Python

Firma	Modo de instanciación	Descripción funcional	Método de MotorTriaVet
`analizar_caso/10`	`(+Especie, +EdadMeses, +Sexo, +Entrada, -Validos, -Ignorados, -Alarmas, -Diagnosticos, -Prioridad, -Recomendacion)`	Consulta completa: valida y normaliza la entrada, infiere y descarta diagnósticos, calcula severidad, certeza, sustento y agravantes, ordena por urgencia y determina la prioridad global. Nunca falla: con datos inválidos responde `datos_invalidos`.	`analizar_caso()`
`diagnosticar_todos/5`	`(+Especie, +EdadMeses, +Sexo, +Entrada, -Diagnosticos)`	Recolección principal: lista única, ordenada y sin duplicados de todos los diagnósticos posibles (`setof/3`). Valor por defecto: `[sin_diagnostico_concluyente]`.	`diagnosticar_todos()`
`catalogo_hallazgos/2`	`(?Grupo, -Hallazgos)`	Hallazgos válidos agrupados por sistema o tipo de antecedente.	`catalogo()`
`nombre_grupo/2`	`(?Grupo, ?Nombre)`	Hecho: nombre legible de cada grupo.	`catalogo()`
`nombre_diagnostico/2`	`(?Diagnostico, ?Nombre)`	Hecho: nombre legible de cada diagnóstico.	`nombre_diagnostico()`

Nota. Cada elemento de `Diagnosticos` es `[Id, Nombre, Severidad, Certeza, Indicacion, Sustento, Agravantes]`. Elaboración propia.

Tabla 9

Principales Predicados Internos

Firma	Modo de instanciación	Descripción funcional
`conclusiones/2`	`(+Caso, -Diagnosticos)`	`setof/3` sobre `diagnostico_posible/2`, con valores por defecto.
`diagnosticos_detallados/2`	`(+Caso, -Detalles)`	Detalla cada conclusión y la ordena por severidad y certeza.
`severidad/3`	`(+Caso, +Diagnostico, -Nivel)`	Crítico si hay alarma; si no, nivel base más agravantes.
`agravantes/3`	`(+Caso, +Diagnostico, -Lista)`	Factores que agravan un cuadro agudo; `[]` para los crónicos.
`toxicidad_para/3`	`(+Sustancia, +Especie, -Nivel)`	Toxicidad registrada o valor por defecto (alto).
`certeza/3`	`(+Caso, +Diagnostico, -Porcentaje)`	Factor de certeza en porcentaje.
`sustento/3`	`(+Caso, +Diagnostico, -Hallazgos)`	Hallazgos del caso que respaldan el diagnóstico.
`prioridad_caso/3`	`(+Caso, +Detalles, -Prioridad)`	Indeterminado, crítico por alarma o el nivel máximo.

Firma	Modo de instanciación	Descripción funcional
nivel_maximo/2	(+Detalles, -Nivel)	Recorre la lista de detalles y devuelve el nivel mayor.
diagnostico_posible/2	(+Caso, ?Diagnostico)	Candidato que no está excluido por otro diagnóstico.
candidato/2	(+Caso, ?Diagnostico)	Hipótesis diagnóstica de Nivel 3.
excluido/2	(+Caso, +Diagnostico)	Verdadero si un diagnóstico dominante también es candidato.
cumple_perfil/2	(+Caso, +Diagnostico)	Especie compatible, requisitos completos y apoyo mínimo.
signo_de_alarma/1	(+Caso)	Hay un signo de alarma directo o signos de shock.
deshidratacion/1	(+Caso)	Dos o más indicadores de pérdida de líquidos.
paciente_vulnerable/1	(+Caso)	Cachorro, geriátrico, gestante o con enfermedad crónica previa.
etapa_vida/3	(+Especie, +EdadMeses, ?Etapa)	Cachorro, geriátrico o adulto (valor por defecto).
normalizar_entrada/3	(+Entrada, -Validos, -Ignorados)	Ordena, elimina duplicados y separa válidos de ignorados.
particionar/3	(+Lista, -Validos, -Invalidos)	Separa los hallazgos reconocidos de los desconocidos.
contiene_todos/2	(+Requeridos, +Lista)	Los requeridos son un subconjunto de la lista.
contar_presentes/3	(+Lista, +Hallazgos, -N)	Cuántos elementos de la lista aparecen en los hallazgos.
interseccion/3	(+Lista, +Hallazgos, -Comunes)	Elementos comunes, en el orden de la primera lista.

Nota. El diccionario completo, con los 55 predicados de reglas y los 23 de hechos, está en `docs/diccionario_predicados.md`. Elaboración propia.

Script de Pruebas por Consola y Evidencia

El script `pruebas.py` ejecuta diez casos de prueba contra la base de conocimiento a través de `motor.py`: los cinco obligatorios y cinco complementarios. Para cada caso imprime en la terminal el objetivo, la entrada, la consulta en Prolog, el resultado esperado, el resultado obtenido y el veredicto. Guarda además la salida completa en `evidencia/salida_pruebas.txt` y termina con código 0 solo si todas las pruebas pasan. En la ejecución registrada aprobaron 10 de 10 pruebas.

Las pruebas se ejecutaron en la terminal integrada de Visual Studio Code mediante las tareas definidas en `.vscode/tasks.json`, desde el menú Terminal > Run Task. Cada caso

obligatorio tiene su propia tarea, que ejecuta `python pruebas.py` con el número del caso. Debajo de la tabla de cada caso se incluye la captura de pantalla de su ejecución (Figuras 8 a 12).

Casos Obligatorios

Las Tablas 10 a 14 documentan los cinco casos que exige la actividad, y las Figuras 8 a 12 muestran su ejecución en la terminal.

Tabla 10

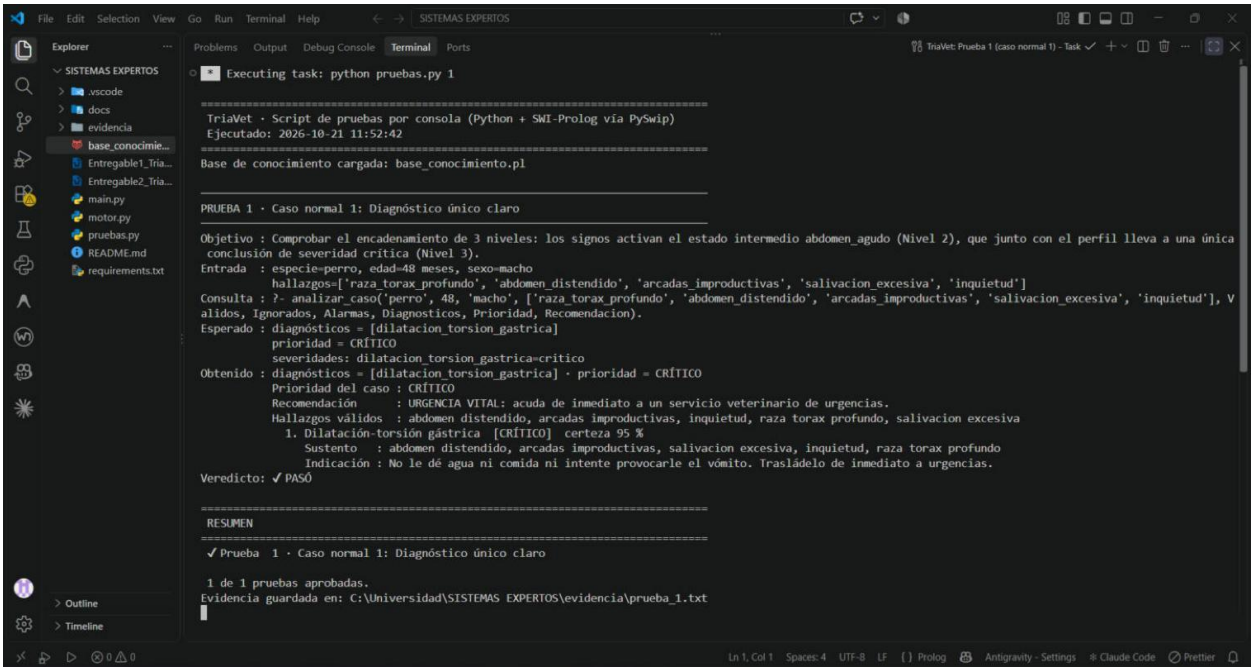
Prueba 1. Caso normal 1: Diagnóstico único claro

Objetivo	Comprobar el encadenamiento de 3 niveles: los signos activan el estado intermedio abdomen_agudo (Nivel 2), que junto con el perfil lleva a una única conclusión de severidad crítica (Nivel 3).
Entrada	especie = perro; edad = 48 meses; sexo = macho; hallazgos = ['raza_torax_profundo', 'abdomen_distendido', 'arcadas_improductivas', 'salivacion_excesiva', 'inquietud']
Consulta en Prolog	`?- analizar_caso('perro', 48, 'macho', ['raza_torax_profundo', 'abdomen_distendido', 'arcadas_improductivas', 'salivacion_excesiva', 'inquietud'], Validos, Ignorados, Alarmas, Diagnosticos, Prioridad, Recomendacion).`
Resultado esperado	diagnósticos = [dilatacion_torsion_gastrica] prioridad = CRÍTICO severidades: dilatacion_torsion_gastrica=critico
Resultado obtenido	Diagnósticos: [dilatacion_torsion_gastrica] Prioridad: CRÍTICO dilatacion_torsion_gastrica: severidad critico, certeza 95 %
Veredicto	**PASÓ**

Nota. Datos obtenidos al ejecutar `pruebas.py`.

Figura 8

Ejecución de la Prueba 1 en la Terminal de Visual Studio Code



Nota. Captura de pantalla tomada al ejecutar la tarea «TriaVet: Prueba 1», que corre `python pruebas.py 1`.

Tabla 11

Prueba 2. Caso normal 2: Múltiples diagnósticos y recolección

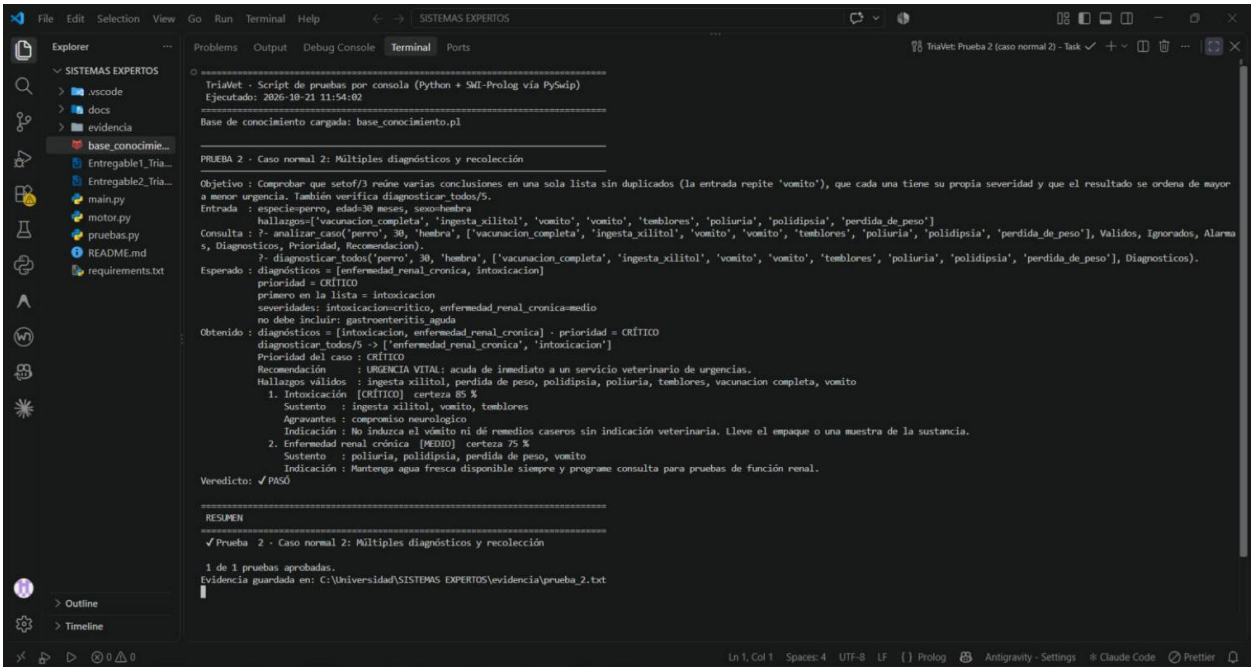
Objetivo	Comprobar que setof/3 reúne varias conclusiones en una sola lista sin duplicados (la entrada repite 'vomito'), que cada una tiene su propia severidad y que el resultado se ordena de mayor a menor urgencia. También verifica diagnosticar_todos/5.
Entrada	especie = perro; edad = 30 meses; sexo = hembra; hallazgos = ['vacunacion_completa', 'ingesta_xilitol', 'vomito', 'vomito', 'temblores', 'poliuria', 'polidipsia', 'perdida_de_peso']
Consulta en Prolog	`?- analizar_caso('perro', 30, 'hembra', ['vacunacion_completa', 'ingesta_xilitol', 'vomito', 'vomito', 'temblores', 'poliuria', 'polidipsia', 'perdida_de_peso'], Validos, Ignorados, Alarmas, Diagnosticos, Prioridad, Recomendacion).` `?- diagnosticar_todos('perro', 30, 'hembra', ['vacunacion_completa', 'ingesta_xilitol', 'vomito', 'vomito', 'temblores', 'poliuria', 'polidipsia', 'perdida_de_peso'], Diagnosticos).`
Resultado esperado	diagnósticos = [enfermedad_renal_cronica, intoxicacion] prioridad = CRÍTICO primero en la lista = intoxicacion severidades: intoxicacion=critico, enfermedad_renal_cronica=medio no debe incluir: gastroenteritis_aguda

Resultado obtenido	Diagnósticos: [intoxicacion, enfermedad_renal_cronica] Prioridad: CRÍTICO intoxicacion: severidad critico, certeza 85 %, agravantes ['compromiso_neurologico'] enfermedad_renal_cronica: severidad medio, certeza 75 % diagnosticar_todos/5: ['enfermedad_renal_cronica', 'intoxicacion']
Veredicto	**PASÓ**

Nota. Datos obtenidos al ejecutar `pruebas.py`.

Figura 9

Ejecución de la Prueba 2 en la Terminal de Visual Studio Code



Nota. Captura de pantalla tomada al ejecutar la tarea «TriaVet: Prueba 2», que corre `python pruebas.py 2`.

Tabla 12

Prueba 3. Caso de exclusión: Negación por fallo (\+) y corte (!)

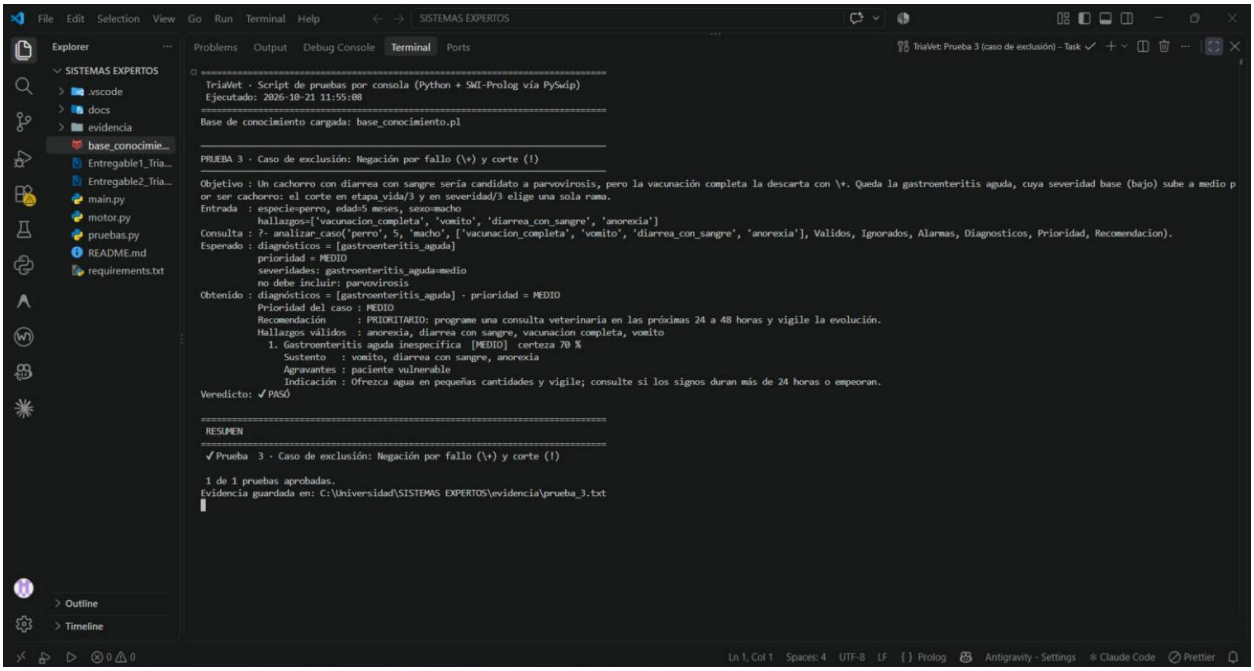
Objetivo	Un cachorro con diarrea con sangre sería candidato a parvovirosis, pero la vacunación completa la descarta con \+. Queda la gastroenteritis aguda, cuya severidad base (bajo) sube a medio por ser cachorro: el corte en etapa_vida/3 y en severidad/3 elige una sola rama.
Entrada	especie = perro; edad = 5 meses; sexo = macho; hallazgos = ['vacunacion_completa', 'vomito', 'diarrea_con_sangre', 'anorexia']
Consulta en Prolog	`?- analizar_caso('perro', 5, 'macho', ['vacunacion_completa', 'vomito', 'diarrea_con_sangre', 'anorexia'], Validos, Ignorados, Alarmas, Diagnosticos, Prioridad, Recomendacion).`

Resultado esperado	diagnósticos = [gastroenteritis_aguda] prioridad = MEDIO severidades: gastroenteritis_aguda=medio no debe incluir: parvovirosis
Resultado obtenido	Diagnósticos: [gastroenteritis_aguda] Prioridad: MEDIO gastroenteritis_aguda: severidad medio, certeza 70 %, agravantes ['paciente_vulnerable']
Veredicto	**PASÓ**

Nota. Datos obtenidos al ejecutar `pruebas.py`.

Figura 10

Ejecución de la Prueba 3 en la Terminal de Visual Studio Code



Nota. Captura de pantalla tomada al ejecutar la tarea «TriaVet: Prueba 3», que corre `python pruebas.py 3`.

Tabla 13

Prueba 4. Caso borde 1: Datos insuficientes / lista parcial

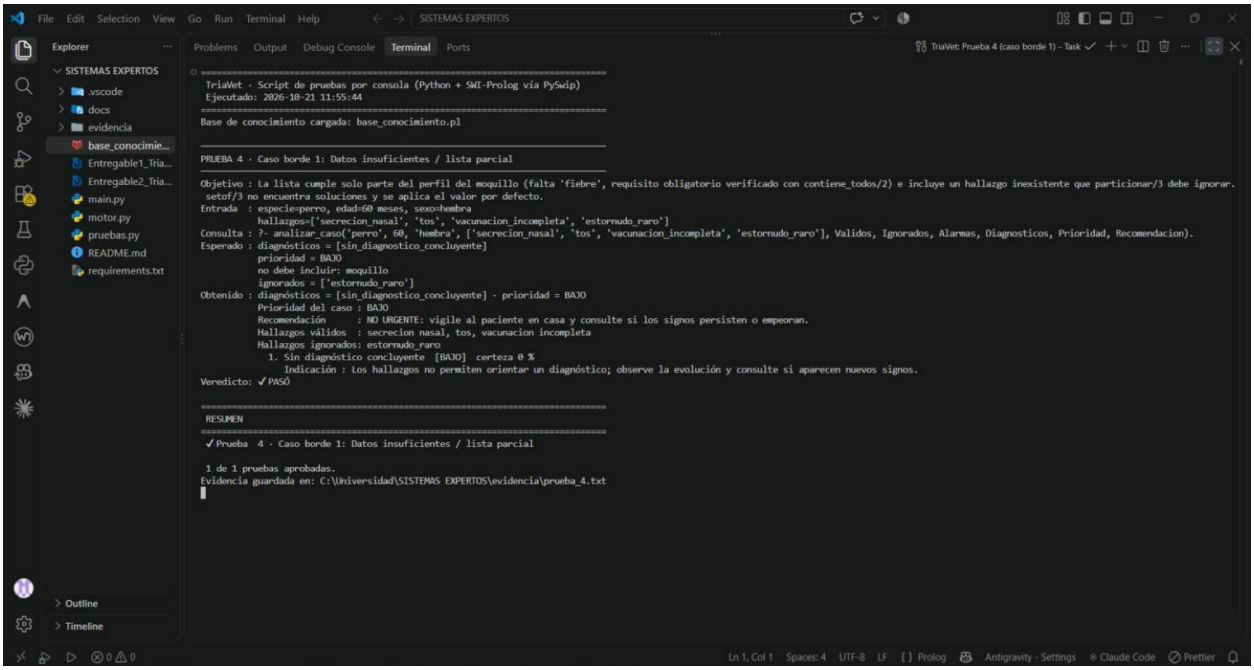
Objetivo	La lista cumple solo parte del perfil del moquillo (falta 'fiebre', requisito obligatorio verificado con contiene_todos/2) e incluye un hallazgo inexistente que particionar/3 debe ignorar. setof/3 no encuentra soluciones y se aplica el valor por defecto.
Entrada	especie = perro; edad = 60 meses; sexo = hembra; hallazgos = ['secrecion_nasal', 'tos', 'vacunacion_incompleta', 'estornudo_raro']
Consulta en Prolog	`?- analizar_caso('perro', 60, 'hembra', ['secrecion_nasal', 'tos', 'vacunacion_incompleta', 'estornudo_raro'], Validos, Ignorados, Alarmas, Diagnosticos, Prioridad, Recomendacion).`

Resultado esperado	diagnósticos = [sin_diagnostico_concluyente] prioridad = BAJO no debe incluir: moquillo ignorados = ['estornudo_raro']
Resultado obtenido	Diagnósticos: [sin_diagnostico_concluyente] Prioridad: BAJO sin_diagnostico_concluyente: severidad bajo, certeza 0 % Ignorados: ['estornudo_raro']
Veredicto	**PASÓ**

Nota. Datos obtenidos al ejecutar `pruebas.py`.

Figura 11

Ejecución de la Prueba 4 en la Terminal de Visual Studio Code



Nota. Captura de pantalla tomada al ejecutar la tarea «TriaVet: Prueba 4», que corre `python pruebas.py 4`.

Tabla 14

Prueba 5. Caso borde 2: Lista vacía sin errores en Python

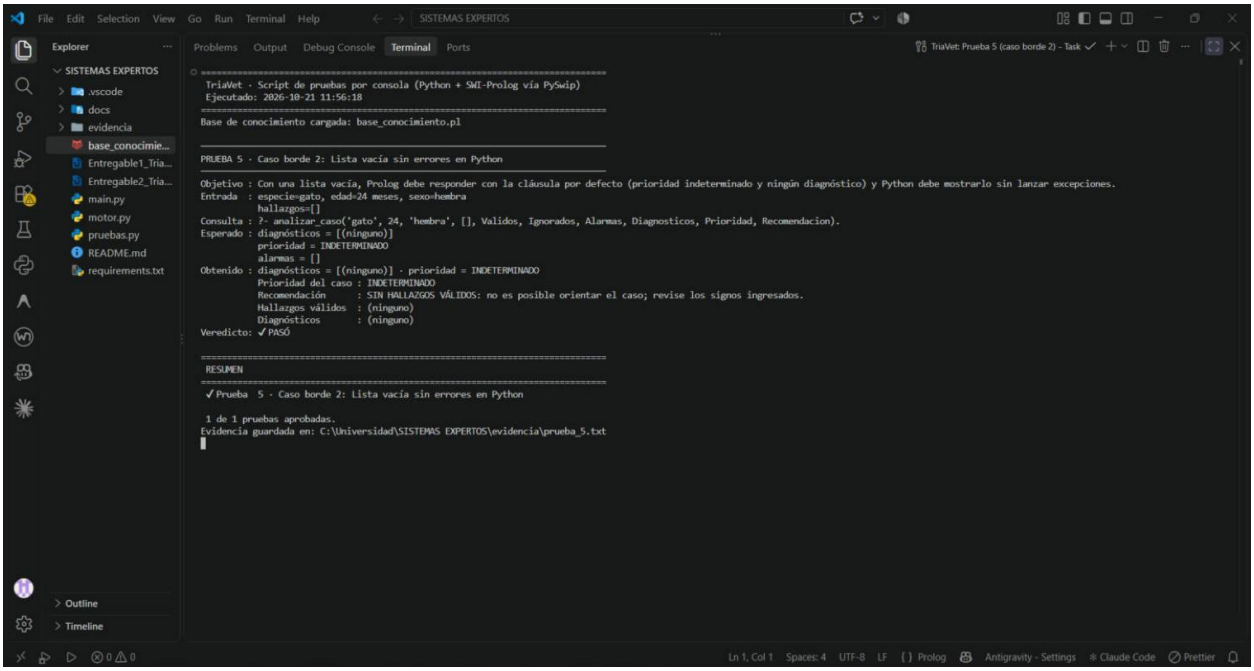
Objetivo	Con una lista vacía, Prolog debe responder con la cláusula por defecto (prioridad indeterminado y ningún diagnóstico) y Python debe mostrarlo sin lanzar excepciones.
Entrada	especie = gato; edad = 24 meses; sexo = hembra; hallazgos = []
Consulta en Prolog	`?- analizar_caso('gato', 24, 'hembra', [], Validos, Ignorados, Alarmas, Diagnosticos, Prioridad, Recomendacion).`

Resultado esperado	diagnósticos = [(ninguno)] prioridad = INDETERMINADO alarmas = []
Resultado obtenido	Diagnósticos: [ninguno] Prioridad: INDETERMINADO
Veredicto	**PASÓ**

Nota. Datos obtenidos al ejecutar `pruebas.py`.

Figura 12

Ejecución de la Prueba 5 en la Terminal de Visual Studio Code



Nota. Captura de pantalla tomada al ejecutar la tarea «TriaVet: Prueba 5», que corre `python pruebas.py 5`.

Casos Complementarios

Los casos de la Tabla 15 refuerzan la verificación de las exclusiones, de la regla de alarma y del manejo de datos inválidos.

Tabla 15

Resultados de los Casos Complementarios

Prueba	Propósito	Resultado obtenido	Veredicto
6	Contraste del caso 3 sin vacunación	Diagnósticos: [parvovirosis] Prioridad: CRÍTICO	PASÓ
7	Exclusión entre diagnósticos (gato macho)	Diagnósticos: [obstruccion_uretral] Prioridad: CRÍTICO	PASÓ
8	Mismo gato sin retención urinaria	Diagnósticos: [cistitis] Prioridad: MEDIO	PASÓ
9	Signo de alarma sin diagnóstico	Diagnósticos: [sin_diagnostico_concluyente] Prioridad: CRÍTICO	PASÓ
10	Datos del paciente no válidos	Diagnósticos: [ninguno] Prioridad: DATOS NO VÁLIDOS	PASÓ

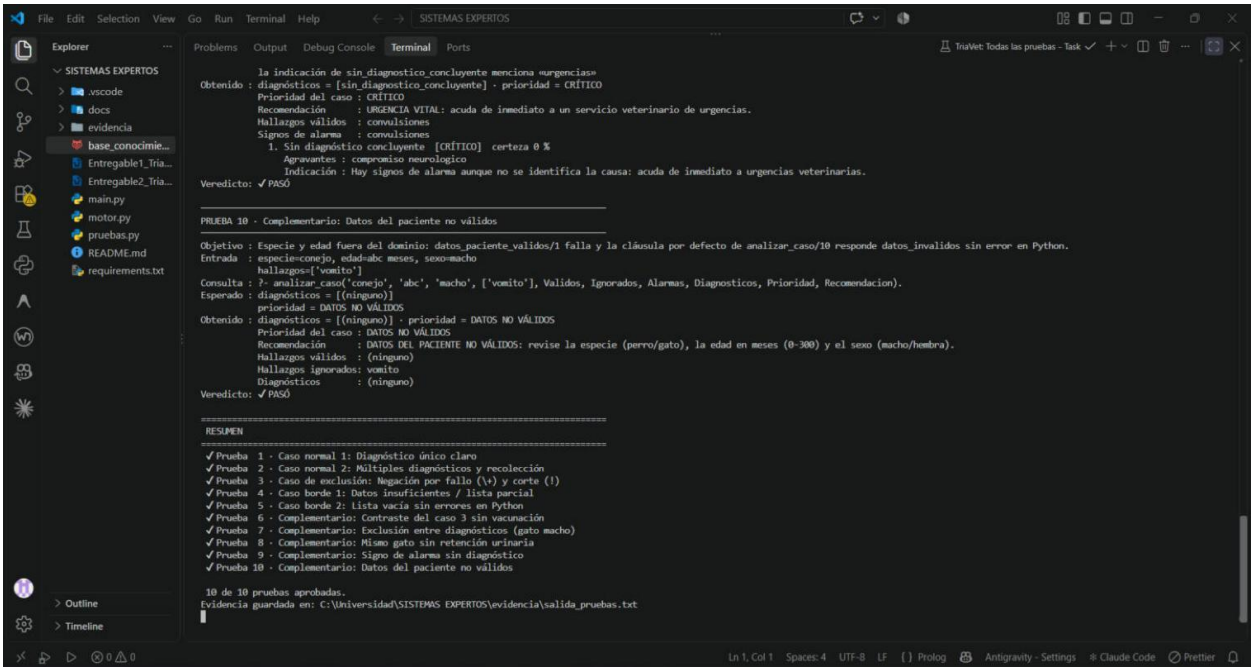
Nota. El detalle de cada caso, incluida la consulta en Prolog, está en el Apéndice E.

Evidencia de Ejecución

La Figura 13 muestra el final de la ejecución completa de los diez casos en la terminal de Visual Studio Code, con el resumen de resultados. La salida completa se encuentra en el Apéndice E.

Figura 13

Resumen de la Ejecución Completa de pruebas.py en Visual Studio Code



Nota. Captura de pantalla tomada al ejecutar la tarea «TriaVet: Todas las pruebas». La salida completa está en `evidencia/salida_pruebas.txt`.

Referencias

Bratko, I. (2012). *Prolog programming for artificial intelligence* (4.ª ed.). Pearson Education.

Forbes Colombia. (2024, 28 de diciembre). *Colombia es el segundo país de Latam en el que más creció el mercado de alimentos para mascotas: estudio*. https://forbes.co/2024/12/28/actualidad/colombia-es-el-segundo-pais-de-latam-en-el-que-mas-crecio-el-mercado-de-alimentos-para-mascotas-estudio/

Giarratano, J. C. y Riley, G. D. (2005). *Expert systems: Principles and programming* (4.ª ed.). Thomson Course Technology.

Ley 1774 de 2016. Por medio de la cual se modifican el Código Civil, la Ley 84 de 1989, el Código Penal, el Código de Procedimiento Penal y se dictan otras disposiciones. 6 de enero de 2016.

Ruys, L. J., Gunning, M., Teske, E., Robben, J. H. y Sigrist, N. E. (2012). Evaluation of a veterinary triage list modified from a human five-point triage system in 485 dogs and cats. *Journal of Veterinary Emergency and Critical Care, 22*(3), 303–312. https://doi.org/10.1111/j.1476-4431.2012.00736.x

Shortliffe, E. H. y Buchanan, B. G. (1975). A model of inexact reasoning in medicine. *Mathematical Biosciences, 23*(3–4), 351–379. https://doi.org/10.1016/0025-5564(75)90047-4

Tekol, Y. (s. f.). *PySwip* (Versión 0.3.3) [Software]. https://pyswip.org

Wielemaker, J., Schrijvers, T., Triska, M. y Lager, T. (2012). SWI-Prolog. *Theory and Practice of Logic Programming, 12*(1–2), 67–96. https://doi.org/10.1017/S1471068411000494

Apéndice A

Código Fuente de base_conocimiento.pl

```
/*  ============================================================================
    TriaVet - Sistema experto para el triage y la orientación diagnóstica
              preliminar de urgencias en perros y gatos

    Archivo     : base_conocimiento.pl
    Motor       : SWI-Prolog 9.x / 10.x
    Integración : Python mediante PySwip (ver motor.py)

    ESTRUCTURA DEL ARCHIVO
      Sección 1. Hechos de la base de conocimiento
      Sección 2. Utilidades de procesamiento de listas
      Sección 3. Reglas de Nivel 1 - validación preliminar
      Sección 4. Reglas de Nivel 2 - inferencia intermedia
      Sección 5. Reglas de Nivel 3 - conclusión diagnóstica
      Sección 6. Severidad, certeza y prioridad
      Sección 7. Meta-predicados de recolección y API para Python

    REPRESENTACIÓN DE UN CASO
      caso(Especie, EdadMeses, Sexo, Hallazgos)
        Especie   : perro | gato
        EdadMeses : entero entre 0 y 300
        Sexo      : macho | hembra
        Hallazgos : lista ordenada y sin duplicados de átomos válidos
                    (signos clínicos y antecedentes del paciente)

    GRAFO DE DEPENDENCIAS (prevención de ciclos)
      Las reglas solo consultan reglas de un nivel igual o inferior y ninguna
      regla se invoca a sí misma por la izquierda:
        Nivel 3 (candidato/2, excluido/2, diagnostico_posible/2)
          -> Nivel 2 (estados intermedios)
            -> Nivel 1 (validación) -> utilidades de listas -> hechos
      excluido/2 llama a candidato/2, nunca a diagnostico_posible/2, por lo
      que aunque existieran exclusiones mutuas no se produciría un bucle.
      Los predicados recursivos sobre listas consumen la cabeza antes de la
      llamada recursiva, de modo que siempre terminan.

    AVISO: herramienta académica de orientación. No reemplaza el examen
    clínico ni el diagnóstico de un médico veterinario.
    ============================================================================ */

:- encoding(utf8).


/*  ============================================================================
    SECCIÓN 1. HECHOS DE LA BASE DE CONOCIMIENTO
    ============================================================================ */

/* ----------------------------------------------------------------------------
   1.1 Pacientes admitidos
   especie_paciente(?Especie, ?Nombre)
   sexo_paciente(?Sexo, ?Nombre)
   ---------------------------------------------------------------------------- */
especie_paciente(perro, 'Perro').
especie_paciente(gato,  'Gato').

sexo_paciente(macho,  'Macho').
sexo_paciente(hembra, 'Hembra').

/* ----------------------------------------------------------------------------
   1.2 Catálogo de signos clínicos observables por el tutor
   signo(?Signo, ?Sistema)
   ---------------------------------------------------------------------------- */
% Sistema digestivo
signo(vomito,                digestivo).
signo(vomito_persistente,    digestivo).
```

```
signo(diarrea,                digestivo).
signo(diarrea_con_sangre,     digestivo).
signo(arcadas_improductivas,  digestivo).
signo(abdomen_distendido,     digestivo).
signo(dolor_abdominal,        digestivo).
signo(salivacion_excesiva,    digestivo).
signo(anorexia,               digestivo).
signo(sin_defecar,            digestivo).
% Sistema urinario
signo(esfuerzo_para_orinar,   urinario).
signo(ausencia_de_orina,      urinario).
signo(miccion_frecuente,      urinario).
signo(sangre_en_orina,        urinario).
signo(poliuria,               urinario).
signo(polidipsia,             urinario).
signo(lamido_genital,         urinario).
% Sistema respiratorio
signo(jadeo_excesivo,         respiratorio).
signo(dificultad_respiratoria, respiratorio).
signo(secrecion_nasal,        respiratorio).
signo(tos,                    respiratorio).
% Sistema nervioso
signo(convulsiones,           neurologico).
signo(temblores,              neurologico).
signo(desorientacion,         neurologico).
signo(mioclonias,             neurologico).
% Estado general
signo(letargo,                general).
signo(fiebre,                 general).
signo(encias_palidas,         general).
signo(encias_rojo_intenso,    general).
signo(llenado_capilar_lento,  general).
signo(colapso,                general).
signo(perdida_de_peso,        general).
signo(halitosis_uremica,      general).
signo(ojos_hundidos,          general).
signo(piel_sin_elasticidad,   general).
signo(inquietud,              general).
signo(vocalizacion_dolor,     general).

/* ------------------------------------------------------------------------
   1.3 Antecedentes y factores de riesgo del paciente
   antecedente(?Antecedente, ?Tipo)
   ------------------------------------------------------------------------ */
antecedente(vacunacion_completa,      vacunacion).
antecedente(vacunacion_incompleta,    vacunacion).
antecedente(gestante,                 condicion).
antecedente(enfermedad_cronica_previa, condicion).
antecedente(raza_torax_profundo,      raza).
antecedente(raza_braquicefala,        raza).
antecedente(exposicion_calor,         exposicion).
antecedente(ingesta_objeto,           exposicion).
antecedente(ingesta_chocolate,        exposicion).
antecedente(ingesta_xilitol,          exposicion).
antecedente(ingesta_uvas,             exposicion).
antecedente(ingesta_cebolla,          exposicion).
antecedente(ingesta_ibuprofeno,       exposicion).
antecedente(ingesta_paracetamol,      exposicion).
antecedente(ingesta_lirio,            exposicion).
antecedente(ingesta_raticida,         exposicion).

/* ------------------------------------------------------------------------
   1.4 Nombres legibles de los grupos de hallazgos (para el menú de consola)
   nombre_grupo(?Grupo, ?Nombre)
   ------------------------------------------------------------------------ */
nombre_grupo(digestivo,    'Signos digestivos').
nombre_grupo(urinario,     'Signos urinarios').
nombre_grupo(respiratorio, 'Signos respiratorios').
nombre_grupo(neurologico,  'Signos neurológicos').
nombre_grupo(general,      'Estado general').
nombre_grupo(vacunacion,   'Vacunación').
```

```
nombre_grupo(condicion,    'Condición del paciente').
nombre_grupo(raza,         'Raza').
nombre_grupo(exposicion,   'Exposiciones e ingestas').

/* ------------------------------------------------------------------------
   1.5 Signos de alarma: elevan la prioridad a crítica sin importar el
   diagnóstico presuntivo
   signo_alarma(?Signo, ?Descripcion)
   ------------------------------------------------------------------------ */
signo_alarma(dificultad_respiratoria, 'dificultad respiratoria').
signo_alarma(convulsiones,            'convulsiones').
signo_alarma(colapso,                 'colapso o incapacidad para levantarse').

/* ------------------------------------------------------------------------
   1.6 Indicadores que definen los estados intermedios del Nivel 2
   indicadores(?Estado, ?ListaDeHallazgos)
   ------------------------------------------------------------------------ */
indicadores(cuadro_digestivo,         [vomito, vomito_persistente, diarrea,
diarrea_con_sangre]).
indicadores(deshidratacion,           [vomito_persistente, diarrea, diarrea_con_sangre,
                                       ojos_hundidos, piel_sin_elasticidad]).
indicadores(compromiso_neurologico,   [convulsiones, temblores, desorientacion,
mioclonias]).
indicadores(depresion_estado_general, [letargo, colapso]).
indicadores(perfusion_deficiente,     [encias_palidas, llenado_capilar_lento]).
indicadores(condicion_vulnerable,     [gestante, enfermedad_cronica_previa]).

/* ------------------------------------------------------------------------
   1.7 Umbrales de edad por especie (en meses)
   umbral_edad(?Especie, ?Etapa, ?Meses)
     cachorro  : edad < Meses        geriatrico : edad >= Meses
   ------------------------------------------------------------------------ */
umbral_edad(perro, cachorro,   12).
umbral_edad(perro, geriatrico, 96).
umbral_edad(gato,  cachorro,   12).
umbral_edad(gato,  geriatrico, 120).

/* ------------------------------------------------------------------------
   1.8 Sustancias tóxicas
   sustancia(?Ingesta, ?Sustancia)
   toxicidad(?Sustancia, ?Especie, ?Nivel)
   Las combinaciones no registradas se tratan con el nivel por defecto de
   intoxicacion (principio de precaución), ver toxicidad_para/3.
   ------------------------------------------------------------------------ */
sustancia(ingesta_chocolate,   chocolate).
sustancia(ingesta_xilitol,     xilitol).
sustancia(ingesta_uvas,        uvas).
sustancia(ingesta_cebolla,     cebolla).
sustancia(ingesta_ibuprofeno,  ibuprofeno).
sustancia(ingesta_paracetamol, paracetamol).
sustancia(ingesta_lirio,       lirio).
sustancia(ingesta_raticida,    raticida).

toxicidad(chocolate,   perro, alto).
toxicidad(chocolate,   gato,  alto).
toxicidad(xilitol,     perro, critico).
toxicidad(uvas,        perro, alto).
toxicidad(cebolla,     perro, medio).
toxicidad(cebolla,     gato,  alto).
toxicidad(ibuprofeno,  perro, alto).
toxicidad(ibuprofeno,  gato,  critico).
toxicidad(paracetamol, perro, alto).
toxicidad(paracetamol, gato,  critico).
toxicidad(lirio,       gato,  critico).
toxicidad(raticida,    perro, alto).
toxicidad(raticida,    gato,  alto).

/* ------------------------------------------------------------------------
   1.9 Catálogo de diagnósticos
   nombre_diagnostico(?Diagnostico, ?Nombre)
   especie_afectada(?Diagnostico, ?Especie)
```

```
206     curso(?Diagnostico, ?Curso)          Curso: agudo | cronico
207     ------------------------------------------------------------------------ */
208  nombre_diagnostico(parvovirosis,                'Parvovirosis canina').
209  nombre_diagnostico(moquillo,                    'Moquillo canino (distemper)').
210  nombre_diagnostico(dilatacion_torsion_gastrica, 'Dilatación-torsión gástrica').
211  nombre_diagnostico(obstruccion_uretral,         'Obstrucción uretral felina').
212  nombre_diagnostico(cistitis,                    'Cistitis / infección urinaria baja').
213  nombre_diagnostico(intoxicacion,                'Intoxicación').
214  nombre_diagnostico(golpe_de_calor,              'Golpe de calor').
215  nombre_diagnostico(obstruccion_intestinal,      'Obstrucción intestinal por cuerpo
     extraño').
216  nombre_diagnostico(enfermedad_renal_cronica,    'Enfermedad renal crónica').
217  nombre_diagnostico(gastroenteritis_aguda,       'Gastroenteritis aguda inespecífica').
218  nombre_diagnostico(sin_diagnostico_concluyente, 'Sin diagnóstico concluyente').
219
220  especie_afectada(parvovirosis,                perro).
221  especie_afectada(moquillo,                    perro).
222  especie_afectada(dilatacion_torsion_gastrica, perro).
223  especie_afectada(obstruccion_uretral,         gato).
224  especie_afectada(cistitis,                    perro).
225  especie_afectada(cistitis,                    gato).
226  especie_afectada(intoxicacion,                perro).
227  especie_afectada(intoxicacion,                gato).
228  especie_afectada(golpe_de_calor,              perro).
229  especie_afectada(golpe_de_calor,              gato).
230  especie_afectada(obstruccion_intestinal,      perro).
231  especie_afectada(obstruccion_intestinal,      gato).
232  especie_afectada(enfermedad_renal_cronica,    perro).
233  especie_afectada(enfermedad_renal_cronica,    gato).
234  especie_afectada(gastroenteritis_aguda,       perro).
235  especie_afectada(gastroenteritis_aguda,       gato).
236
237  curso(parvovirosis,                agudo).
238  curso(moquillo,                    agudo).
239  curso(dilatacion_torsion_gastrica, agudo).
240  curso(obstruccion_uretral,         agudo).
241  curso(cistitis,                    agudo).
242  curso(intoxicacion,                agudo).
243  curso(golpe_de_calor,              agudo).
244  curso(obstruccion_intestinal,      agudo).
245  curso(enfermedad_renal_cronica,    cronico).
246  curso(gastroenteritis_aguda,       agudo).
247  curso(sin_diagnostico_concluyente, agudo).
248
249  /* ------------------------------------------------------------------------
250     1.10 Perfiles clínicos
251     requisito(?Diagnostico, ?ListaObligatoria)   todos deben estar presentes
252     apoyo(?Diagnostico, ?ListaDeApoyo)           refuerzan la hipótesis
253     minimo_apoyo(?Diagnostico, ?N)               cuántos de apoyo se exigen
254     certeza_base(?Diagnostico, ?Factor)          certeza inicial (0..1)
255     ------------------------------------------------------------------------ */
256  requisito(parvovirosis,                [diarrea_con_sangre]).
257  requisito(moquillo,                    [secrecion_nasal, fiebre]).
258  requisito(dilatacion_torsion_gastrica, [abdomen_distendido, arcadas_improductivas]).
259  requisito(obstruccion_uretral,         [esfuerzo_para_orinar, ausencia_de_orina]).
260  requisito(cistitis,                    []).
261  requisito(intoxicacion,                []).
262  requisito(golpe_de_calor,              [exposicion_calor, jadeo_excesivo]).
263  requisito(obstruccion_intestinal,      [vomito_persistente]).
264  requisito(enfermedad_renal_cronica,    [poliuria, polidipsia]).
265  requisito(gastroenteritis_aguda,       []).
266  requisito(sin_diagnostico_concluyente, []).
267
268  apoyo(parvovirosis,                [vomito, vomito_persistente, letargo, anorexia, fiebre,
269                                      vacunacion_incompleta]).
270  apoyo(moquillo,                    [tos, mioclonias, convulsiones, letargo, anorexia,
     diarrea,
271                                      vacunacion_incompleta]).
272  apoyo(dilatacion_torsion_gastrica, [salivacion_excesiva, inquietud, dolor_abdominal,
273                                      raza_torax_profundo, colapso]).
274  apoyo(obstruccion_uretral,         [vocalizacion_dolor, lamido_genital, letargo, vomito,
```

```
                                    anorexia]).
apoyo(cistitis,                    [miccion_frecuente, sangre_en_orina,
esfuerzo_para_orinar,
                                    lamido_genital, vocalizacion_dolor]).
apoyo(intoxicacion,                [vomito, salivacion_excesiva, temblores, convulsiones,
                                    letargo, diarrea, desorientacion]).
apoyo(golpe_de_calor,              [encias_rojo_intenso, colapso, letargo, vomito,
                                    desorientacion, raza_braquicefala,
dificultad_respiratoria]).
apoyo(obstruccion_intestinal,      [ingesta_objeto, sin_defecar, dolor_abdominal, anorexia,
                                    letargo]).
apoyo(enfermedad_renal_cronica,    [perdida_de_peso, halitosis_uremica, vomito, anorexia,
                                    letargo]).
apoyo(gastroenteritis_aguda,       [vomito, diarrea, diarrea_con_sangre, anorexia]).
apoyo(sin_diagnostico_concluyente, []).

minimo_apoyo(parvovirosis,                1).
minimo_apoyo(moquillo,                    1).
minimo_apoyo(dilatacion_torsion_gastrica, 1).
minimo_apoyo(obstruccion_uretral,         0).
minimo_apoyo(cistitis,                    2).
minimo_apoyo(intoxicacion,                0).
minimo_apoyo(golpe_de_calor,              1).
minimo_apoyo(obstruccion_intestinal,      2).
minimo_apoyo(enfermedad_renal_cronica,    1).
minimo_apoyo(gastroenteritis_aguda,       1).
minimo_apoyo(sin_diagnostico_concluyente, 0).

certeza_base(parvovirosis,                0.60).
certeza_base(moquillo,                    0.50).
certeza_base(dilatacion_torsion_gastrica, 0.70).
certeza_base(obstruccion_uretral,         0.75).
certeza_base(cistitis,                    0.55).
certeza_base(intoxicacion,                0.65).
certeza_base(golpe_de_calor,              0.65).
certeza_base(obstruccion_intestinal,      0.50).
certeza_base(enfermedad_renal_cronica,    0.55).
certeza_base(gastroenteritis_aguda,       0.40).
certeza_base(sin_diagnostico_concluyente, 0.00).

/* ------------------------------------------------------------------------
   1.11 Exclusiones entre diagnósticos
   excluye(?Dominante, ?Excluido)
   Si el diagnóstico Dominante es candidato, el Excluido se descarta porque
   el primero explica mejor (y con mayor urgencia) los mismos hallazgos.
   ------------------------------------------------------------------------ */
excluye(obstruccion_uretral,         cistitis).
excluye(parvovirosis,                gastroenteritis_aguda).
excluye(intoxicacion,                gastroenteritis_aguda).
excluye(obstruccion_intestinal,      gastroenteritis_aguda).
excluye(dilatacion_torsion_gastrica, gastroenteritis_aguda).

/* ------------------------------------------------------------------------
   1.12 Escala de severidad
   severidad_base(?Diagnostico, ?Nivel)
   nivel_valor(?Nivel, ?Peso)           ponderación numérica para ordenar
   siguiente_nivel(?Nivel, ?NivelSuperior)
   recomendacion(?Prioridad, ?Texto)
   ------------------------------------------------------------------------ */
severidad_base(parvovirosis,                alto).
severidad_base(moquillo,                    alto).
severidad_base(dilatacion_torsion_gastrica, critico).
severidad_base(obstruccion_uretral,         critico).
severidad_base(cistitis,                    medio).
severidad_base(intoxicacion,                alto).
severidad_base(golpe_de_calor,              critico).
severidad_base(obstruccion_intestinal,      alto).
severidad_base(enfermedad_renal_cronica,    medio).
severidad_base(gastroenteritis_aguda,       bajo).
severidad_base(sin_diagnostico_concluyente, bajo).

```

```
nivel_valor(bajo,    1).
nivel_valor(medio,   2).
nivel_valor(alto,    3).
nivel_valor(critico, 4).

siguiente_nivel(bajo,    medio).
siguiente_nivel(medio,   alto).
siguiente_nivel(alto,    critico).
siguiente_nivel(critico, critico).

recomendacion(critico,
    'URGENCIA VITAL: acuda de inmediato a un servicio veterinario de urgencias.').
recomendacion(alto,
    'URGENTE: busque atención veterinaria hoy mismo, en las próximas horas.').
recomendacion(medio,
    'PRIORITARIO: programe una consulta veterinaria en las próximas 24 a 48 horas y vigile la evolución.').
recomendacion(bajo,
    'NO URGENTE: vigile al paciente en casa y consulte si los signos persisten o empeoran.').
recomendacion(indeterminado,
    'SIN HALLAZGOS VÁLIDOS: no es posible orientar el caso; revise los signos ingresados.').
recomendacion(datos_invalidos,
    'DATOS DEL PACIENTE NO VÁLIDOS: revise la especie (perro/gato), la edad en meses (0-300) y el sexo (macho/hembra).').

/* ------------------------------------------------------------------------
   1.13 Indicaciones mientras se consigue atención veterinaria
   indicacion(?Clave, ?Texto)
   Clave es un diagnóstico o la clave especial alarma_sin_diagnostico.
   ------------------------------------------------------------------------ */
indicacion(parvovirosis,
    'Aísle al cachorro de otros perros, no lo obligue a comer y llévelo hoy mismo al veterinario: la deshidratación avanza rápido.').
indicacion(moquillo,
    'Aísle al perro de otros perros y acuda a consulta; si aparecen convulsiones, vaya a urgencias.').
indicacion(dilatacion_torsion_gastrica,
    'No le dé agua ni comida ni intente provocarle el vómito. Trasládelo de inmediato a urgencias.').
indicacion(obstruccion_uretral,
    'No espere a que orine por sí solo ni le presione el abdomen. Trasládelo de inmediato a urgencias.').
indicacion(cistitis,
    'Asegure agua fresca y arenero o paseos frecuentes; programe consulta. Si deja de orinar, acuda a urgencias.').
indicacion(intoxicacion,
    'No induzca el vómito ni dé remedios caseros sin indicación veterinaria. Lleve el empaque o una muestra de la sustancia.').
indicacion(golpe_de_calor,
    'Llévelo a la sombra y refrésquelo con agua a temperatura ambiente (no helada) mientras lo traslada a urgencias.').
indicacion(obstruccion_intestinal,
    'No le dé comida ni laxantes. Acuda hoy al veterinario e informe qué objeto pudo ingerir.').
indicacion(enfermedad_renal_cronica,
    'Mantenga agua fresca disponible siempre y programe consulta para pruebas de función renal.').
indicacion(gastroenteritis_aguda,
    'Ofrezca agua en pequeñas cantidades y vigile; consulte si los signos duran más de 24 horas o empeoran.').
indicacion(sin_diagnostico_concluyente,
    'Los hallazgos no permiten orientar un diagnóstico; observe la evolución y consulte si aparecen nuevos signos.').
indicacion(alarma_sin_diagnostico,
    'Hay signos de alarma aunque no se identifica la causa: acuda de inmediato a urgencias veterinarias.').


/*  ========================================================================
    SECCIÓN 2. UTILIDADES DE PROCESAMIENTO DE LISTAS
```

```
400      Todas recorren la lista consumiendo la cabeza antes de la llamada
401      recursiva (recursión por la derecha), por lo que siempre terminan.
402      ========================================================================== */
403
404  %!  presente(+Hallazgo, +Hallazgos) is semidet.
405  %   Verdadero si Hallazgo está en la lista. memberchk/2 es determinista:
406  %   no deja puntos de elección que generen soluciones repetidas.
407  presente(Hallazgo, Hallazgos) :-
408      memberchk(Hallazgo, Hallazgos).
409
410  %!  alguno_presente(+Candidatos, +Hallazgos) is semidet.
411  %   Verdadero si al menos un elemento de Candidatos está en Hallazgos.
412  %   CORTE: basta el primer elemento encontrado; sin el corte, el
413  %   backtracking produciría una solución por cada coincidencia.
414  alguno_presente(Candidatos, Hallazgos) :-
415      member(X, Candidatos),
416      presente(X, Hallazgos), !.
417
418  %!  contiene_todos(+Requeridos, +Hallazgos) is semidet.
419  %   Verifica que Requeridos sea un subconjunto de Hallazgos.
420  contiene_todos([], _).
421  contiene_todos([R|Rs], Hallazgos) :-
422      presente(R, Hallazgos),
423      contiene_todos(Rs, Hallazgos).
424
425  %!  contar_presentes(+Lista, +Hallazgos, -N) is det.
426  %   Cuenta cuántos elementos de Lista aparecen en Hallazgos.
427  %   CORTE: la segunda y la tercera cláusula son mutuamente excluyentes
428  %   (el elemento está o no está); el corte impide que, al retroceder, la
429  %   tercera cláusula vuelva a contar un elemento que sí estaba presente.
430  contar_presentes([], _, 0).
431  contar_presentes([X|Xs], Hallazgos, N) :-
432      presente(X, Hallazgos), !,
433      contar_presentes(Xs, Hallazgos, N0),
434      N is N0 + 1.
435  contar_presentes([_|Xs], Hallazgos, N) :-
436      contar_presentes(Xs, Hallazgos, N).
437
438  %!  interseccion(+Lista, +Hallazgos, -Comunes) is det.
439  %   Filtra los elementos de Lista que están en Hallazgos (conserva el orden).
440  %   CORTE: mismo criterio de exclusión mutua que contar_presentes/3.
441  interseccion([], _, []).
442  interseccion([X|Xs], Hallazgos, [X|Comunes]) :-
443      presente(X, Hallazgos), !,
444      interseccion(Xs, Hallazgos, Comunes).
445  interseccion([_|Xs], Hallazgos, Comunes) :-
446      interseccion(Xs, Hallazgos, Comunes).
447
448  %!  particionar(+Lista, -Validos, -Invalidos) is det.
449  %   Separa los hallazgos reconocidos por la base de conocimiento de los
450  %   que no lo son (errores de digitación, signos fuera del dominio).
451  %   CORTE: un elemento es válido o inválido, nunca ambas cosas.
452  particionar([], [], []).
453  particionar([H|T], [H|Validos], Invalidos) :-
454      hallazgo_valido(H), !,
455      particionar(T, Validos, Invalidos).
456  particionar([H|T], Validos, [H|Invalidos]) :-
457      particionar(T, Validos, Invalidos).
458
459  %!  pares_valores(+Pares, -Valores) is det.
460  %   Extrae los valores de una lista de pares Clave-Valor.
461  pares_valores([], []).
462  pares_valores([_-V|Pares], [V|Valores]) :-
463      pares_valores(Pares, Valores).
464
465
466  /*  ============================================================================
467      SECCIÓN 3. REGLAS DE NIVEL 1 - VALIDACIÓN PRELIMINAR
468      Clasifican los datos de entrada y verifican condiciones elementales.
469      ========================================================================== */
470
```

```
%!  hallazgo_valido(+Hallazgo) is semidet.
%   Un hallazgo es válido si es un átomo registrado como signo o antecedente.
%   CORTE: si ya es un signo, no se busca además entre los antecedentes.
hallazgo_valido(H) :-
    atom(H),
    signo(H, _), !.
hallazgo_valido(H) :-
    atom(H),
    antecedente(H, _).

%!  grupo_hallazgo(?Hallazgo, ?Grupo) is nondet.
%   Asocia cada hallazgo con su grupo (sistema orgánico o tipo de antecedente).
grupo_hallazgo(H, Grupo) :-
    signo(H, Grupo).
grupo_hallazgo(H, Grupo) :-
    antecedente(H, Grupo).

%!  normalizar_entrada(+Entrada, -Validos, -Ignorados) is det.
%   Ordena la lista recibida desde Python, elimina duplicados (sort/2) y la
%   separa en hallazgos válidos e ignorados.
normalizar_entrada(Entrada, Validos, Ignorados) :-
    sort(Entrada, SinDuplicados),
    particionar(SinDuplicados, Validos, Ignorados).

%!  datos_paciente_validos(+Caso) is semidet.
%   Verifica especie, sexo y edad (entero entre 0 y 300 meses).
datos_paciente_validos(caso(Especie, Edad, Sexo, _)) :-
    especie_paciente(Especie, _),
    sexo_paciente(Sexo, _),
    integer(Edad),
    Edad >= 0,
    Edad =< 300.

%!  etapa_vida(+Especie, +EdadMeses, ?Etapa) is det.
%   Clasifica la edad en cachorro, adulto o geriatrico.
%   CORTE: las tres cláusulas son mutuamente excluyentes y la última es el
%   valor por defecto (fallback). La unificación de Etapa se hace DESPUÉS
%   del corte para que la regla sea correcta también cuando se consulta con
%   Etapa ya instanciada (p. ej. etapa_vida(perro, 5, adulto) debe fallar).
etapa_vida(Especie, Edad, Etapa) :-
    umbral_edad(Especie, cachorro, Meses),
    Edad < Meses, !,
    Etapa = cachorro.
etapa_vida(Especie, Edad, Etapa) :-
    umbral_edad(Especie, geriatrico, Meses),
    Edad >= Meses, !,
    Etapa = geriatrico.
etapa_vida(_, _, adulto).

%!  etapa(+Caso, ?Etapa) is det.
etapa(caso(Especie, Edad, _, _), Etapa) :-
    etapa_vida(Especie, Edad, Etapa).


/*  ============================================================================
    SECCIÓN 4. REGLAS DE NIVEL 2 - INFERENCIA INTERMEDIA
    Combinan reglas de Nivel 1 y hechos para deducir estados intermedios que
    el usuario no ingresa directamente.
    ============================================================================ */

%!  cuadro_digestivo(+Caso) is semidet.
cuadro_digestivo(caso(_, _, _, H)) :-
    indicadores(cuadro_digestivo, Indicadores),
    alguno_presente(Indicadores, H).

%!  deshidratacion(+Caso) is semidet.
%   Se infiere con al menos dos indicadores de pérdida de líquidos.
deshidratacion(caso(_, _, _, H)) :-
    indicadores(deshidratacion, Indicadores),
    contar_presentes(Indicadores, H, N),
    N >= 2.
```

```
542
543 %!  compromiso_neurologico(+Caso) is semidet.
544 compromiso_neurologico(caso(_, _, _, H)) :-
545     indicadores(compromiso_neurologico, Indicadores),
546     alguno_presente(Indicadores, H).
547
548 %!  signos_de_shock(+Caso) is semidet.
549 %   Depresión del estado general + perfusión deficiente.
550 signos_de_shock(caso(_, _, _, H)) :-
551     indicadores(depresion_estado_general, Depresion),
552     alguno_presente(Depresion, H),
553     indicadores(perfusion_deficiente, Perfusion),
554     alguno_presente(Perfusion, H).
555
556 %!  signo_de_alarma(+Caso) is semidet.
557 %   CORTE: basta un signo de alarma directo; solo si no hay ninguno se
558 %   evalúa el estado intermedio de shock (segunda cláusula).
559 signo_de_alarma(caso(_, _, _, H)) :-
560     member(S, H),
561     signo_alarma(S, _), !.
562 signo_de_alarma(Caso) :-
563     signos_de_shock(Caso).
564
565 %!  alarmas_presentes(+Caso, -Alarmas) is det.
566 %   Lista los signos de alarma directos y, si aplica, el estado de shock.
567 alarmas_presentes(Caso, Alarmas) :-
568     Caso = caso(_, _, _, H),
569     findall(S, (member(S, H), signo_alarma(S, _)), Directas),
570     alarma_por_shock(Caso, PorShock),
571     append(Directas, PorShock, Alarmas).
572
573 %   CORTE + FALLBACK: si hay shock se devuelve [signos_de_shock]; en caso
574 %   contrario la cláusula por defecto devuelve la lista vacía.
575 alarma_por_shock(Caso, [signos_de_shock]) :-
576     signos_de_shock(Caso), !.
577 alarma_por_shock(_, []).
578
579 %!  abdomen_agudo(+Caso) is semidet.
580 abdomen_agudo(caso(_, _, _, H)) :-
581     alguno_presente([dolor_abdominal, abdomen_distendido], H),
582     alguno_presente([vomito_persistente, arcadas_improductivas], H).
583
584 %!  retencion_urinaria(+Caso) is semidet.
585 retencion_urinaria(caso(_, _, _, H)) :-
586     contiene_todos([esfuerzo_para_orinar, ausencia_de_orina], H).
587
588 %!  exposicion_toxica(+Caso, -Sustancia) is nondet.
589 %   Una solución por cada sustancia tóxica reportada.
590 exposicion_toxica(caso(_, _, _, H), Sustancia) :-
591     member(Ingesta, H),
592     sustancia(Ingesta, Sustancia).
593
594 %!  hipertermia_probable(+Caso) is semidet.
595 hipertermia_probable(caso(_, _, _, H)) :-
596     contiene_todos([exposicion_calor, jadeo_excesivo], H).
597
598 %!  sin_proteccion_vacunal(+Caso) is semidet.
599 %   NEGACIÓN POR FALLO: el paciente se considera sin protección si NO se
600 %   reporta vacunación completa. H está instanciada (viene del caso).
601 sin_proteccion_vacunal(caso(_, _, _, H)) :-
602     \+ presente(vacunacion_completa, H).
603
604 %!  susceptible_parvovirus(+Caso) is semidet.
605 %   Sin protección vacunal y, además, cachorro o con esquema incompleto.
606 %   CORTE: si ya es cachorro no hace falta revisar la segunda cláusula.
607 susceptible_parvovirus(Caso) :-
608     sin_proteccion_vacunal(Caso),
609     etapa(Caso, cachorro), !.
610 susceptible_parvovirus(Caso) :-
611     sin_proteccion_vacunal(Caso),
612     Caso = caso(_, _, _, H),
```

```
613     presente(vacunacion_incompleta, H).
614
615 %!  paciente_vulnerable(+Caso) is semidet.
616 %   Cachorro, geriátrico, gestante o con enfermedad crónica previa.
617 %   CORTE: una sola condición de vulnerabilidad es suficiente.
618 paciente_vulnerable(Caso) :-
619     etapa(Caso, Etapa),
620     memberchk(Etapa, [cachorro, geriatrico]), !.
621 paciente_vulnerable(caso(_, _, _, H)) :-
622     indicadores(condicion_vulnerable, Condiciones),
623     alguno_presente(Condiciones, H).
624
625 %!  cumple_perfil(+Caso, +Diagnostico) is semidet.
626 %   El caso cumple el perfil si la especie es compatible, están TODOS los
627 %   requisitos (subconjunto) y hay al menos el mínimo de signos de apoyo.
628 cumple_perfil(caso(Especie, _, _, H), Diagnostico) :-
629     especie_afectada(Diagnostico, Especie),
630     requisito(Diagnostico, Requisitos),
631     contiene_todos(Requisitos, H),
632     apoyo(Diagnostico, Apoyo),
633     contar_presentes(Apoyo, H, N),
634     minimo_apoyo(Diagnostico, Minimo),
635     N >= Minimo.
636
637
638 /*  ========================================================================
639     SECCIÓN 5. REGLAS DE NIVEL 3 - CONCLUSIÓN DIAGNÓSTICA
640     candidato/2 integra reglas de Nivel 2 para plantear hipótesis;
641     diagnostico_posible/2 aplica los criterios de descarte.
642     ======================================================================== */
643
644 %!  candidato(+Caso, ?Diagnostico) is nondet.
645 %   Hipótesis diagnóstica antes de aplicar las exclusiones entre diagnósticos.
646
647 % Parvovirosis: cuadro digestivo en un perro susceptible.
648 % La negación de la vacunación completa está dentro de susceptible_parvovirus/1.
649 candidato(Caso, parvovirosis) :-
650     cuadro_digestivo(Caso),
651     susceptible_parvovirus(Caso),
652     cumple_perfil(Caso, parvovirosis).
653
654 % Moquillo: secreción nasal + fiebre en un perro sin protección vacunal.
655 candidato(Caso, moquillo) :-
656     sin_proteccion_vacunal(Caso),
657     cumple_perfil(Caso, moquillo).
658
659 % Dilatación-torsión gástrica: abdomen agudo con distensión y arcadas.
660 candidato(Caso, dilatacion_torsion_gastrica) :-
661     abdomen_agudo(Caso),
662     cumple_perfil(Caso, dilatacion_torsion_gastrica).
663
664 % Obstrucción uretral: retención urinaria en un gato macho.
665 candidato(Caso, obstruccion_uretral) :-
666     Caso = caso(gato, _, macho, _),
667     retencion_urinaria(Caso),
668     cumple_perfil(Caso, obstruccion_uretral).
669
670 % Cistitis: al menos dos signos urinarios bajos (puede quedar excluida por
671 % la obstrucción uretral, ver excluye/2).
672 candidato(Caso, cistitis) :-
673     cumple_perfil(Caso, cistitis).
674
675 % Intoxicación: basta con que se reporte la ingesta de una sustancia tóxica.
676 % once/1 evita una solución repetida por cada sustancia ingerida.
677 candidato(Caso, intoxicacion) :-
678     once(exposicion_toxica(Caso, _)),
679     cumple_perfil(Caso, intoxicacion).
680
681 % Golpe de calor: hipertermia probable y al menos un signo de apoyo.
682 candidato(Caso, golpe_de_calor) :-
683     hipertermia_probable(Caso),
```

```
684      cumple_perfil(Caso, golpe_de_calor).
685
686  % Obstrucción intestinal: vómito persistente y dos signos de apoyo.
687  candidato(Caso, obstruccion_intestinal) :-
688      cuadro_digestivo(Caso),
689      cumple_perfil(Caso, obstruccion_intestinal).
690
691  % Enfermedad renal crónica: poliuria + polidipsia en un paciente que NO es
692  % cachorro. NEGACIÓN POR FALLO con Especie y Edad ya instanciadas.
693  candidato(Caso, enfermedad_renal_cronica) :-
694      Caso = caso(Especie, Edad, _, _),
695      \+ etapa_vida(Especie, Edad, cachorro),
696      cumple_perfil(Caso, enfermedad_renal_cronica).
697
698  % Gastroenteritis aguda inespecífica: cuadro digestivo SIN signos de alarma.
699  % NEGACIÓN POR FALLO: si hay cualquier signo de alarma el cuadro no puede
700  % considerarse una gastroenteritis inespecífica.
701  candidato(Caso, gastroenteritis_aguda) :-
702      cuadro_digestivo(Caso),
703      \+ signo_de_alarma(Caso),
704      cumple_perfil(Caso, gastroenteritis_aguda).
705
706  %!  excluido(+Caso, +Diagnostico) is semidet.
707  %   Un diagnóstico se excluye si otro diagnóstico dominante es candidato.
708  %   CORTE: basta con encontrar una exclusión.
709  %   Llama a candidato/2 y no a diagnostico_posible/2: así se evitan ciclos.
710  excluido(Caso, Diagnostico) :-
711      excluye(Dominante, Diagnostico),
712      candidato(Caso, Dominante), !.
713
714  %!  diagnostico_posible(+Caso, ?Diagnostico) is nondet.
715  %   NEGACIÓN POR FALLO: Diagnostico ya está instanciado por candidato/2
716  %   cuando se evalúa \+ excluido/2.
717  diagnostico_posible(Caso, Diagnostico) :-
718      candidato(Caso, Diagnostico),
719      \+ excluido(Caso, Diagnostico).
720
721
722  /*  ============================================================================
723      SECCIÓN 6. SEVERIDAD, CERTEZA Y PRIORIDAD
724      ============================================================================ */
725
726  %!  severidad(+Caso, +Diagnostico, -Nivel) is det.
727  %   1) Un signo de alarma fija el nivel en critico.
728  %   2) En otro caso: nivel base del diagnóstico (o de la sustancia tóxica),
729  %      subido un nivel por cada factor agravante.
730  %   CORTE: la primera cláusula, cuando aplica, anula la segunda.
731  severidad(Caso, _, Nivel) :-
732      signo_de_alarma(Caso), !,
733      Nivel = critico.
734  severidad(Caso, Diagnostico, Nivel) :-
735      nivel_base(Caso, Diagnostico, Base),
736      agravantes(Caso, Diagnostico, Agravantes),
737      length(Agravantes, K),
738      subir_niveles(Base, K, Nivel).
739
740  %!  nivel_base(+Caso, +Diagnostico, -Nivel) is det.
741  %   CORTE: la intoxicación tiene su propio cálculo; las demás usan el hecho
742  %   severidad_base/2.
743  nivel_base(Caso, intoxicacion, Nivel) :- !,
744      severidad_intoxicacion(Caso, Nivel).
745  nivel_base(_, Diagnostico, Nivel) :-
746      severidad_base(Diagnostico, Nivel).
747
748  %!  severidad_intoxicacion(+Caso, -Nivel) is det.
749  %   Toma el nivel más alto entre todas las sustancias ingeridas.
750  severidad_intoxicacion(Caso, Nivel) :-
751      Caso = caso(Especie, _, _, _),
752      findall(Peso-N,
753              ( exposicion_toxica(Caso, S),
754                toxicidad_para(S, Especie, N),
```

```
755                nivel_valor(N, Peso) ),
756              Pares),
757      max_member(_-Nivel, Pares).
758
759  %!  toxicidad_para(+Sustancia, +Especie, -Nivel) is det.
760  %   CORTE + FALLBACK: si la combinación sustancia/especie está registrada se
761  %   usa ese nivel; si no, se aplica el nivel por defecto de intoxicacion.
762  toxicidad_para(Sustancia, Especie, Nivel) :-
763      toxicidad(Sustancia, Especie, N), !,
764      Nivel = N.
765  toxicidad_para(_, _, Nivel) :-
766      severidad_base(intoxicacion, Nivel).
767
768  %!  agravantes(+Caso, +Diagnostico, -Lista) is det.
769  %   CORTE: los cuadros crónicos no se agravan por condiciones del paciente;
770  %   para los agudos se recogen todos los agravantes presentes.
771  agravantes(_, Diagnostico, []) :-
772      curso(Diagnostico, cronico), !.
773  agravantes(Caso, _, Lista) :-
774      findall(A, agravante(Caso, A), Lista).
775
776  %!  agravante(+Caso, ?Agravante) is nondet.
777  agravante(Caso, paciente_vulnerable) :-
778      paciente_vulnerable(Caso).
779  agravante(Caso, deshidratacion) :-
780      deshidratacion(Caso).
781  agravante(Caso, compromiso_neurologico) :-
782      compromiso_neurologico(Caso).
783
784  %!  subir_niveles(+Nivel, +K, -NivelFinal) is det.
785  %   Sube K niveles en la escala (critico es el tope).
786  %   CORTE: K = 0 es el caso base y excluye a la cláusula recursiva.
787  subir_niveles(Nivel0, 0, Nivel) :- !,
788      Nivel = Nivel0.
789  subir_niveles(Nivel0, K, Nivel) :-
790      K > 0,
791      siguiente_nivel(Nivel0, Nivel1),
792      K1 is K - 1,
793      subir_niveles(Nivel1, K1, Nivel).
794
795  %!  certeza(+Caso, +Diagnostico, -Porcentaje) is det.
796  %   Factor de certeza simplificado: certeza base + 0.10 por cada signo de
797  %   apoyo presente, con tope de 0.95. Se devuelve en porcentaje entero.
798  certeza(caso(_, _, _, H), Diagnostico, Porcentaje) :-
799      certeza_base(Diagnostico, Base),
800      apoyo(Diagnostico, Apoyo),
801      contar_presentes(Apoyo, H, N),
802      Factor is min(0.95, Base + 0.10 * N),
803      Porcentaje is round(Factor * 100).
804
805  %!  sustento(+Caso, +Diagnostico, -Hallazgos) is det.
806  %   Hallazgos del caso que respaldan el diagnóstico (explicación).
807  %   CORTE: la intoxicación agrega además las ingestas reportadas.
808  sustento(Caso, intoxicacion, Hallazgos) :- !,
809      Caso = caso(_, _, _, H),
810      findall(I, (member(I, H), sustancia(I, _)), Ingestas),
811      perfil_presente(intoxicacion, H, Signos),
812      append(Ingestas, Signos, Hallazgos).
813  sustento(caso(_, _, _, H), Diagnostico, Hallazgos) :-
814      perfil_presente(Diagnostico, H, Hallazgos).
815
816  perfil_presente(Diagnostico, H, Presentes) :-
817      requisito(Diagnostico, Requisitos),
818      apoyo(Diagnostico, Apoyo),
819      append(Requisitos, Apoyo, Todos),
820      interseccion(Todos, H, Presentes).
821
822  %!  nivel_maximo(+Diagnosticos, -Nivel) is det.
823  %   Recorre la lista de diagnósticos detallados y devuelve el nivel mayor.
824  %   CORTE: la lista de un solo elemento es el caso base.
825  nivel_maximo([[_, _, N|_]], Nivel) :- !,
```

```
826     Nivel = N.
827 nivel_maximo([[_, _, Nivel|_]|Resto], Maximo) :-
828     nivel_maximo(Resto, MaximoResto),
829     mayor_nivel(Nivel, MaximoResto, Maximo).
830
831 %   CORTE + FALLBACK: si A no es mayor o igual que B, el resultado es B.
832 mayor_nivel(A, B, Mayor) :-
833     nivel_valor(A, VA),
834     nivel_valor(B, VB),
835     VA >= VB, !,
836     Mayor = A.
837 mayor_nivel(_, B, B).
838
839 %!  prioridad_caso(+Caso, +Diagnosticos, -Prioridad) is det.
840 %   1) Sin hallazgos válidos -> indeterminado.
841 %   2) Con un signo de alarma -> critico (prevalece sobre todo lo demás).
842 %   3) En otro caso -> el nivel más alto entre los diagnósticos.
843 %   CORTE: las cláusulas se evalúan en orden de precedencia y son excluyentes.
844 prioridad_caso(caso(_, _, _, []), _, Prioridad) :- !,
845     Prioridad = indeterminado.
846 prioridad_caso(Caso, _, Prioridad) :-
847     signo_de_alarma(Caso), !,
848     Prioridad = critico.
849 prioridad_caso(_, Diagnosticos, Prioridad) :-
850     nivel_maximo(Diagnosticos, Prioridad).
851
852
853 /*  ========================================================================
854     SECCIÓN 7. META-PREDICADOS DE RECOLECCIÓN Y API PARA PYTHON
855     ======================================================================== */
856
857 %!  preparar_caso(+Especie, +Edad, +Sexo, +Entrada, -Caso, -Ignorados) is semidet.
858 %   Valida la entrada recibida desde Python y construye el término caso/4.
859 preparar_caso(Especie, Edad, Sexo, Entrada, Caso, Ignorados) :-
860     is_list(Entrada),
861     normalizar_entrada(Entrada, Validos, Ignorados),
862     Caso = caso(Especie, Edad, Sexo, Validos),
863     datos_paciente_validos(Caso).
864
865 %!  conclusiones(+Caso, -Diagnosticos) is det.
866 %   RECOLECCIÓN con setof/3: agrupa en una sola lista ordenada y SIN
867 %   DUPLICADOS todos los diagnósticos posibles del caso.
868 %   CORTE + FALLBACK:
869 %     - sin hallazgos válidos no se evalúa nada (lista vacía);
870 %     - setof/3 falla si no hay soluciones; en ese caso la última cláusula
871 %       devuelve el valor por defecto [sin_diagnostico_concluyente].
872 conclusiones(caso(_, _, _, []), []) :- !.
873 conclusiones(Caso, Diagnosticos) :-
874     setof(D, diagnostico_posible(Caso, D), Diagnosticos), !.
875 conclusiones(_, [sin_diagnostico_concluyente]).
876
877 %!  detallar(+Caso, +Diagnostico, -Detalle) is det.
878 %   Detalle = [Id, Nombre, Severidad, Certeza, Indicacion, Sustento, Agravantes]
879 detallar(Caso, D, [D, Nombre, Nivel, Certeza, Indicacion, Sustento, Agravantes]) :-
880     nombre_diagnostico(D, Nombre),
881     severidad(Caso, D, Nivel),
882     certeza(Caso, D, Certeza),
883     indicacion_para(Caso, D, Indicacion),
884     sustento(Caso, D, Sustento),
885     agravantes(Caso, D, Agravantes).
886
887 %!  indicacion_para(+Caso, +Diagnostico, -Texto) is det.
888 %   CORTE + FALLBACK: si no hay diagnóstico pero sí signos de alarma, la
889 %   indicación genérica de "observar" sería peligrosa; se usa la de alarma.
890 %   En cualquier otro caso se usa la indicación propia del diagnóstico.
891 indicacion_para(Caso, sin_diagnostico_concluyente, Texto) :-
892     signo_de_alarma(Caso), !,
893     indicacion(alarma_sin_diagnostico, Texto).
894 indicacion_para(_, Diagnostico, Texto) :-
895     indicacion(Diagnostico, Texto).
896
```

```
%!  diagnosticos_detallados(+Caso, -Detalles) is det.
%   Detalla cada conclusión y ordena de mayor a menor severidad y certeza.
diagnosticos_detallados(Caso, Ordenados) :-
    conclusiones(Caso, Diagnosticos),
    findall(Detalle,
            ( member(D, Diagnosticos),
              once(detallar(Caso, D, Detalle)) ),
            Detalles),
    ordenar_por_prioridad(Detalles, Ordenados).

ordenar_por_prioridad(Detalles, Ordenados) :-
    findall(Clave-Detalle,
            ( member(Detalle, Detalles),
              clave_orden(Detalle, Clave) ),
            Pares),
    sort(1, @>=, Pares, ParesOrdenados),
    pares_valores(ParesOrdenados, Ordenados).

clave_orden([_, _, Nivel, Certeza|_], Clave) :-
    nivel_valor(Nivel, Peso),
    Clave is Peso * 1000 + Certeza.

%!  catalogo_hallazgos(?Grupo, -Hallazgos) is nondet.
%   RECOLECCIÓN con setof/3 agrupando por Grupo: devuelve una solución por
%   cada grupo con la lista ordenada de sus hallazgos (menú de consola).
catalogo_hallazgos(Grupo, Hallazgos) :-
    setof(H, grupo_hallazgo(H, Grupo), Hallazgos).

%!  diagnosticar_todos(+Especie, +Edad, +Sexo, +Entrada, -Diagnosticos) is det.
%   PREDICADO PRINCIPAL DE RECOLECCIÓN: lista única y sin duplicados de
%   todas las conclusiones para el caso.
%   CORTE + FALLBACK: si los datos del paciente no son válidos se devuelve []
%   en lugar de fallar, para que Python nunca reciba un error.
diagnosticar_todos(Especie, Edad, Sexo, Entrada, Diagnosticos) :-
    preparar_caso(Especie, Edad, Sexo, Entrada, Caso, _), !,
    conclusiones(Caso, Diagnosticos).
diagnosticar_todos(_, _, _, _, []).

%!  analizar_caso(+Especie, +Edad, +Sexo, +Entrada,
%!                -Validos, -Ignorados, -Alarmas, -Diagnosticos,
%!                -Prioridad, -Recomendacion) is det.
%   Consulta completa invocada desde Python en una sola llamada.
%   CORTE + FALLBACK: si la entrada o los datos del paciente no son válidos
%   se responde con la prioridad datos_invalidos, sin lanzar errores.
analizar_caso(Especie, Edad, Sexo, Entrada,
              Validos, Ignorados, Alarmas, Diagnosticos, Prioridad, Recomendacion) :-
    preparar_caso(Especie, Edad, Sexo, Entrada, Caso, Ignorados), !,
    Caso = caso(_, _, _, Validos),
    alarmas_presentes(Caso, Alarmas),
    diagnosticos_detallados(Caso, Diagnosticos),
    prioridad_caso(Caso, Diagnosticos, Prioridad),
    recomendacion(Prioridad, Recomendacion).
analizar_caso(_, _, _, Entrada, [], Ignorados, [], [], datos_invalidos, Recomendacion) :-
    entrada_como_lista(Entrada, Ignorados),
    recomendacion(datos_invalidos, Recomendacion).

%   CORTE + FALLBACK: si la entrada no es una lista se envuelve en una.
entrada_como_lista(Entrada, Entrada) :-
    is_list(Entrada), !.
entrada_como_lista(Entrada, [Entrada]).
```

Apéndice B

Código Fuente de motor.py

```
"""
motor.py - Puente entre Python y la base de conocimiento de TriaVet.

Este módulo carga ``base_conocimiento.pl`` en SWI-Prolog mediante PySwip y
expone funciones que invocan los predicados públicos de Prolog.

Responsabilidad estricta: aquí NO hay lógica de inferencia. Validar los
hallazgos, descartar diagnósticos, calcular la severidad y ordenar los
resultados ocurre en Prolog. Python solo:
  1. convierte los datos de entrada en términos de Prolog,
  2. invoca el predicado,
  3. convierte la respuesta en objetos de Python para mostrarlos.

Predicados de Prolog invocados (ver docs/diccionario_predicados.md):
  analizar_caso/10       -> MotorTriaVet.analizar_caso()
  diagnosticar_todos/5   -> MotorTriaVet.diagnosticar_todos()
  catalogo_hallazgos/2   -> MotorTriaVet.catalogo()
  nombre_grupo/2         -> MotorTriaVet.catalogo()
  nombre_diagnostico/2   -> MotorTriaVet.nombre_diagnostico()
"""

from __future__ import annotations

import re
from dataclasses import dataclass, field
from pathlib import Path
from typing import Iterable

from pyswip import Prolog

RUTA_BASE = Path(__file__).resolve().with_name("base_conocimiento.pl")


class ErrorMotor(RuntimeError):
    """Error al cargar la base de conocimiento o al consultar a Prolog."""


@dataclass
class Diagnostico:
    """Un diagnóstico presuntivo tal como lo devuelve analizar_caso/10."""

    id: str
    nombre: str
    severidad: str
    certeza: int
    indicacion: str
    sustento: list[str] = field(default_factory=list)
    agravantes: list[str] = field(default_factory=list)


@dataclass
class ResultadoCaso:
    """Respuesta completa de analizar_caso/10."""

    consulta: str
    validos: list[str]
    ignorados: list[str]
    alarmas: list[str]
    diagnosticos: list[Diagnostico]
    prioridad: str
    recomendacion: str

    @property
    def ids(self) -> list[str]:
        """Identificadores de los diagnósticos en el orden entregado por Prolog."""
```

```
66          return [d.id for d in self.diagnosticos]
67
68
69  # ------------------------------------------------------------------------
70  # Conversión de valores de Python a términos de Prolog
71  # ------------------------------------------------------------------------
72
73  def _normalizar(valor: object) -> str:
74      """Formato de texto: minúsculas, '_' entre palabras y solo caracteres de palabra.
75
76      Es solo formato (como pasar a minúsculas): decidir si el hallazgo existe
77      le corresponde a Prolog (particionar/3).
78      """
79      texto = "_".join(str(valor).strip().lower().split())
80      return re.sub(r"[^\w]", "", texto)
81
82
83  def termino_atomo(valor: object) -> str:
84      """Convierte un valor en un átomo de Prolog entre comillas simples.
85
86      Las comillas evitan que un texto de usuario se interprete como variable
87      (p. ej. 'Fiebre') o como código Prolog.
88      """
89      return f"'{_normalizar(valor)}'"
90
91
92  def termino_edad(valor: object) -> str:
93      """La edad viaja como entero si lo es; si no, como átomo.
94
95      Python no decide si la edad es válida: si llega un átomo, Prolog la
96      rechaza en datos_paciente_validos/1 y responde con datos_invalidos.
97      """
98      texto = str(valor).strip()
99      return texto if texto.isdigit() else termino_atomo(texto)
100
101
102 def termino_lista(valores: Iterable[object]) -> str:
103     """Convierte una secuencia de Python en una lista de átomos de Prolog."""
104     return "[" + ", ".join(termino_atomo(v) for v in valores) + "]"
105
106
107 # ------------------------------------------------------------------------
108 # Conversión de valores de Prolog a Python
109 # ------------------------------------------------------------------------
110
111 def _texto(valor: object) -> str:
112     if isinstance(valor, bytes):
113         return valor.decode("utf-8")
114     return str(valor)
115
116
117 def _lista_textos(valor: object) -> list[str]:
118     return [_texto(v) for v in valor] if isinstance(valor, list) else []
119
120
121 def _a_diagnostico(fila: list) -> Diagnostico:
122     """Fila de Prolog: [Id, Nombre, Severidad, Certeza, Indicacion, Sustento,
    Agravantes]."""
123     id_, nombre, severidad, certeza, indicacion, sustento, agravantes = fila
124     return Diagnostico(
125         id=_texto(id_),
126         nombre=_texto(nombre),
127         severidad=_texto(severidad),
128         certeza=int(certeza),
129         indicacion=_texto(indicacion),
130         sustento=_lista_textos(sustento),
131         agravantes=_lista_textos(agravantes),
132     )
133
134
135 # ------------------------------------------------------------------------
```

```
# Motor
# ------------------------------------------------------------------------

class MotorTriaVet:
    """Fachada de Python sobre la base de conocimiento de Prolog."""

    def __init__(self, ruta_base: Path | str = RUTA_BASE) -> None:
        ruta = Path(ruta_base).resolve()
        if not ruta.exists():
            raise ErrorMotor(f"No se encontró la base de conocimiento: {ruta}")
        self._prolog = Prolog()
        try:
            # as_posix(): Prolog interpreta '\' como escape dentro de un átomo.
            self._prolog.consult(ruta.as_posix())
        except Exception as exc:  # PrologError u otros errores de carga
            raise ErrorMotor(f"No se pudo cargar {ruta.name}: {exc}") from exc
        self.ruta_base = ruta

    # -- consultas genéricas ---------------------------------------------

    def consultar(self, consulta: str, max_resultados: int = -1) -> list[dict]:
        """Ejecuta una consulta y devuelve todas sus soluciones como diccionarios."""
        try:
            return list(self._prolog.query(consulta, maxresult=max_resultados))
        except Exception as exc:
            raise ErrorMotor(f"Error al consultar «{consulta}»: {exc}") from exc

    # -- construcción de consultas (también las usa pruebas.py) ----------

    @staticmethod
    def consulta_analisis(especie, edad, sexo, hallazgos) -> str:
        return (
            f"analizar_caso({termino_atomo(especie)}, {termino_edad(edad)}, "
            f"{termino_atomo(sexo)}, {termino_lista(hallazgos)}, "
            "Validos, Ignorados, Alarmas, Diagnosticos, Prioridad, Recomendacion)"
        )

    @staticmethod
    def consulta_diagnosticar_todos(especie, edad, sexo, hallazgos) -> str:
        return (
            f"diagnosticar_todos({termino_atomo(especie)}, {termino_edad(edad)}, "
            f"{termino_atomo(sexo)}, {termino_lista(hallazgos)}, Diagnosticos)"
        )

    # -- API pública -----------------------------------------------------

    def analizar_caso(self, especie, edad, sexo, hallazgos: Iterable[object]) ->
ResultadoCaso:
        """Invoca analizar_caso/10 y devuelve la respuesta completa."""
        consulta = self.consulta_analisis(especie, edad, sexo, list(hallazgos))
        soluciones = self.consultar(consulta, max_resultados=1)
        if not soluciones:
            # analizar_caso/10 tiene una cláusula por defecto, así que esto
            # solo ocurriría si la base de conocimiento estuviera incompleta.
            raise ErrorMotor("Prolog no devolvió respuesta para el caso.")
        s = soluciones[0]
        return ResultadoCaso(
            consulta=consulta,
            validos=_lista_textos(s["Validos"]),
            ignorados=_lista_textos(s["Ignorados"]),
            alarmas=_lista_textos(s["Alarmas"]),
            diagnosticos=[_a_diagnostico(f) for f in s["Diagnosticos"]],
            prioridad=_texto(s["Prioridad"]),
            recomendacion=_texto(s["Recomendacion"]),
        )

    def diagnosticar_todos(self, especie, edad, sexo, hallazgos: Iterable[object]) ->
list[str]:
        """Invoca diagnosticar_todos/5: lista única, ordenada y sin duplicados."""
        consulta = self.consulta_diagnosticar_todos(especie, edad, sexo, list(hallazgos))
        soluciones = self.consultar(consulta, max_resultados=1)
```

```
205         return _lista_textos(soluciones[0]["Diagnosticos"]) if soluciones else []
206
207     def catalogo(self) -> list[tuple[str, str, list[str]]]:
208         """Grupos de hallazgos válidos: [(grupo, nombre_legible, [hallazgos...])]."""
209         soluciones = self.consultar(
210             "nombre_grupo(Grupo, Nombre), catalogo_hallazgos(Grupo, Hallazgos)"
211         )
212         return [
213             (_texto(s["Grupo"]), _texto(s["Nombre"]), _lista_textos(s["Hallazgos"]))
214             for s in soluciones
215         ]
216
217     def nombre_diagnostico(self, id_diagnostico: str) -> str:
218         """Nombre legible de un diagnóstico (nombre_diagnostico/2)."""
219         soluciones = self.consultar(
220             f"nombre_diagnostico({termino_atomo(id_diagnostico)}, Nombre)", max_resultados=1
221         )
222         return _texto(soluciones[0]["Nombre"]) if soluciones else id_diagnostico
223
224
225 # ---------------------------------------------------------------------------
226 # Presentación en consola (compartida por main.py y pruebas.py)
227 # ---------------------------------------------------------------------------
228
229 ETIQUETA_NIVEL = {
230     "critico": "CRÍTICO",
231     "alto": "ALTO",
232     "medio": "MEDIO",
233     "bajo": "BAJO",
234     "indeterminado": "INDETERMINADO",
235     "datos_invalidos": "DATOS NO VÁLIDOS",
236 }
237
238
239 def legible(atomo: str) -> str:
240     """'vomito_persistente' -> 'vomito persistente' (solo para mostrar)."""
241     return atomo.replace("_", " ")
242
243
244 def formatear_resultado(resultado: ResultadoCaso, sangria: str = "") -> str:
245     """Texto listo para imprimir con la respuesta de Prolog."""
246     s = sangria
247     lineas = [
248         f"{s}Prioridad del caso : {ETIQUETA_NIVEL.get(resultado.prioridad,
    resultado.prioridad)}",
249         f"{s}Recomendación      : {resultado.recomendacion}",
250         f"{s}Hallazgos válidos  : {', '.join(map(legible, resultado.validos)) or
    '(ninguno)'}",
251     ]
252     if resultado.ignorados:
253         lineas.append(f"{s}Hallazgos ignorados: {', '.join(resultado.ignorados)}")
254     if resultado.alarmas:
255         lineas.append(f"{s}Signos de alarma   : {', '.join(map(legible,
    resultado.alarmas))}")
256     if not resultado.diagnosticos:
257         lineas.append(f"{s}Diagnósticos       : (ninguno)")
258     for i, d in enumerate(resultado.diagnosticos, start=1):
259         lineas.append(
260             f"{s}  {i}. {d.nombre}  [{ETIQUETA_NIVEL.get(d.severidad, d.severidad)}]"
261             f"  certeza {d.certeza} %"
262         )
263         if d.sustento:
264             lineas.append(f"{s}     Sustento   : {', '.join(map(legible, d.sustento))}")
265         if d.agravantes:
266             lineas.append(f"{s}     Agravantes : {', '.join(map(legible, d.agravantes))}")
267         lineas.append(f"{s}     Indicación : {d.indicacion}")
268     return "\n".join(lineas)
```

Apéndice C

Código Fuente de main.py

```
"""
main.py - Consola interactiva de TriaVet.

Uso:
    python main.py

Pide los datos del paciente y los hallazgos observados, los envía a Prolog
(analizar_caso/10) y muestra el resultado. Toda la validación y la
inferencia ocurren en base_conocimiento.pl; este archivo solo lee del
teclado e imprime.
"""

from __future__ import annotations

import sys

from motor import ErrorMotor, MotorTriaVet, formatear_resultado, legible

AVISO = (
    "Aviso: TriaVet es una herramienta académica de orientación. "
    "No reemplaza la valoración de un médico veterinario."
)


def leer(mensaje: str) -> str:
    """input() que trata Ctrl+Z / Ctrl+D como salida limpia."""
    try:
        return input(mensaje).strip()
    except EOFError:
        print()
        raise SystemExit(0)


def mostrar_catalogo(catalogo) -> list[str]:
    """Imprime el menú numerado de hallazgos y devuelve la lista en ese orden."""
    numerados: list[str] = []
    for _, nombre_grupo, hallazgos in catalogo:
        print(f"\n  {nombre_grupo}")
        for hallazgo in hallazgos:
            numerados.append(hallazgo)
            print(f"    {len(numerados):>2}. {legible(hallazgo)}")
    return numerados


def interpretar_seleccion(texto: str, numerados: list[str]) -> list[str]:
    """Traduce '1, 5, fiebre' a nombres de hallazgos.

    Un número del menú se reemplaza por su hallazgo; cualquier otro texto se
    envía tal cual y Prolog decide si es válido (si no, lo reporta como
    ignorado).
    """
    seleccion = []
    for token in (t.strip() for t in texto.split(",")):
        if not token:
            continue
        if token.isdigit() and 1 <= int(token) <= len(numerados):
            seleccion.append(numerados[int(token) - 1])
        else:
            seleccion.append(token)
    return seleccion


def evaluar_caso(motor: MotorTriaVet, numerados: list[str]) -> None:
    print("\n--- Datos del paciente ---")
    especie = leer("Especie (perro/gato): ")
```

```
    edad = leer("Edad en meses: ")
    sexo = leer("Sexo (macho/hembra): ")

    print("\n--- Hallazgos observados ---")
    print("Escriba los números del menú o los nombres, separados por comas.")
    print("Deje la línea vacía si no observó ningún signo.")
    hallazgos = interpretar_seleccion(leer("> "), numerados)

    resultado = motor.analizar_caso(especie, edad, sexo, hallazgos)
    print("\n=== Resultado de TriaVet ===")
    print(formatear_resultado(resultado))
    print(f"\n{AVISO}")


def main() -> int:
    if hasattr(sys.stdout, "reconfigure"):
        sys.stdout.reconfigure(encoding="utf-8")

    print("=" * 72)
    print(" TriaVet - Sistema experto de triage para urgencias en perros y gatos")
    print("=" * 72)
    try:
        motor = MotorTriaVet()
    except ErrorMotor as exc:
        print(f"Error: {exc}")
        return 1

    print("\nHallazgos que reconoce la base de conocimiento:")
    numerados = mostrar_catalogo(motor.catalogo())

    while True:
        try:
            evaluar_caso(motor, numerados)
        except ErrorMotor as exc:
            print(f"\nError al consultar el motor de inferencia: {exc}")
        if leer("\n¿Evaluar otro caso? (s/n): ").lower() not in ("s", "si", "sí"):
            print("Hasta luego.")
            return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Apéndice D

Código Fuente de pruebas.py

```
"""
pruebas.py - Casos de prueba por consola de TriaVet.

Uso:
    python pruebas.py            ejecuta todos los casos
    python pruebas.py 1 3        ejecuta solo los casos indicados

Ejecuta los casos de prueba contra base_conocimiento.pl a través de
motor.py. Para cada caso muestra: objetivo, entrada, consulta en Prolog,
resultado esperado, resultado obtenido y veredicto. La salida también se
guarda en evidencia/salida_pruebas.txt (o en evidencia/prueba_<n>.txt
cuando se ejecutan casos sueltos).

Código de salida: 0 si todas las pruebas pasan, 1 si alguna falla.
"""

from __future__ import annotations

import sys
from dataclasses import dataclass, field
from datetime import datetime
from pathlib import Path

from motor import ETIQUETA_NIVEL, MotorTriaVet, ResultadoCaso, formatear_resultado

CARPETA_EVIDENCIA = Path(__file__).resolve().with_name("evidencia")
ANCHO = 78


@dataclass
class CasoPrueba:
    numero: str
    categoria: str
    titulo: str
    objetivo: str
    especie: object
    edad: object
    sexo: object
    hallazgos: list
    esperado_diagnosticos: list[str]
    esperado_prioridad: str
    # Comprobaciones opcionales
    primero: str | None = None              # diagnóstico que debe ir primero
    no_incluye: list[str] = field(default_factory=list)
    esperado_ignorados: list[str] | None = None
    esperado_alarmas: list[str] | None = None
    severidades: dict[str, str] = field(default_factory=dict)
    indicacion_contiene: dict[str, str] = field(default_factory=dict)
    verificar_diagnosticar_todos: bool = False


CASOS: list[CasoPrueba] = [
    # 1 ------------------------------------------------------------------
    CasoPrueba(
        numero="1",
        categoria="Caso normal 1",
        titulo="Diagnóstico único claro",
        objetivo=(
            "Comprobar el encadenamiento de 3 niveles: los signos activan el "
            "estado intermedio abdomen_agudo (Nivel 2), que junto con el perfil "
            "lleva a una única conclusión de severidad crítica (Nivel 3)."
        ),
        especie="perro", edad=48, sexo="macho",
        hallazgos=["raza_torax_profundo", "abdomen_distendido",
                   "arcadas_improductivas", "salivacion_excesiva", "inquietud"],
```

```
66          esperado_diagnosticos=["dilatacion_torsion_gastrica"],
67          esperado_prioridad="critico",
68          severidades={"dilatacion_torsion_gastrica": "critico"},
69      ),
70      # 2 ------------------------------------------------------------------
71      CasoPrueba(
72          numero="2",
73          categoria="Caso normal 2",
74          titulo="Múltiples diagnósticos y recolección",
75          objetivo=(
76              "Comprobar que setof/3 reúne varias conclusiones en una sola lista "
77              "sin duplicados (la entrada repite 'vomito'), que cada una tiene "
78              "su propia severidad y que el resultado se ordena de mayor a "
79              "menor urgencia. También verifica diagnosticar_todos/5."
80          ),
81          especie="perro", edad=30, sexo="hembra",
82          hallazgos=["vacunacion_completa", "ingesta_xilitol", "vomito", "vomito",
83                     "temblores", "poliuria", "polidipsia", "perdida_de_peso"],
84          esperado_diagnosticos=["enfermedad_renal_cronica", "intoxicacion"],
85          esperado_prioridad="critico",
86          primero="intoxicacion",
87          severidades={"intoxicacion": "critico", "enfermedad_renal_cronica": "medio"},
88          no_incluye=["gastroenteritis_aguda"],
89          verificar_diagnosticar_todos=True,
90      ),
91      # 3 ------------------------------------------------------------------
92      CasoPrueba(
93          numero="3",
94          categoria="Caso de exclusión",
95          titulo="Negación por fallo (\\+) y corte (!)",
96          objetivo=(
97              "Un cachorro con diarrea con sangre sería candidato a parvovirosis, "
98              "pero la vacunación completa la descarta con \\+. Queda la "
99              "gastroenteritis aguda, cuya severidad base (bajo) sube a medio "
100             "por ser cachorro: el corte en etapa_vida/3 y en severidad/3 "
101             "elige una sola rama."
102         ),
103         especie="perro", edad=5, sexo="macho",
104         hallazgos=["vacunacion_completa", "vomito", "diarrea_con_sangre", "anorexia"],
105         esperado_diagnosticos=["gastroenteritis_aguda"],
106         esperado_prioridad="medio",
107         no_incluye=["parvovirosis"],
108         severidades={"gastroenteritis_aguda": "medio"},
109     ),
110     # 4 ------------------------------------------------------------------
111     CasoPrueba(
112         numero="4",
113         categoria="Caso borde 1",
114         titulo="Datos insuficientes / lista parcial",
115         objetivo=(
116             "La lista cumple solo parte del perfil del moquillo (falta 'fiebre', "
117             "requisito obligatorio verificado con contiene_todos/2) e incluye "
118             "un hallazgo inexistente que particionar/3 debe ignorar. setof/3 "
119             "no encuentra soluciones y se aplica el valor por defecto."
120         ),
121         especie="perro", edad=60, sexo="hembra",
122         hallazgos=["secrecion_nasal", "tos", "vacunacion_incompleta", "estornudo_raro"],
123         esperado_diagnosticos=["sin_diagnostico_concluyente"],
124         esperado_prioridad="bajo",
125         no_incluye=["moquillo"],
126         esperado_ignorados=["estornudo_raro"],
127     ),
128     # 5 ------------------------------------------------------------------
129     CasoPrueba(
130         numero="5",
131         categoria="Caso borde 2",
132         titulo="Lista vacía sin errores en Python",
133         objetivo=(
134             "Con una lista vacía, Prolog debe responder con la cláusula por "
135             "defecto (prioridad indeterminado y ningún diagnóstico) y Python "
136             "debe mostrarlo sin lanzar excepciones."
```

```
        ),
        especie="gato", edad=24, sexo="hembra",
        hallazgos=[],
        esperado_diagnosticos=[],
        esperado_prioridad="indeterminado",
        esperado_alarmas=[],
    ),
    # Complementarios ------------------------------------------------------
    CasoPrueba(
        numero="6",
        categoria="Complementario",
        titulo="Contraste del caso 3 sin vacunación",
        objetivo=(
            "Con los mismos signos del caso 3 pero sin vacunación completa, la "
            "parvovirosis sí se concluye y excluye a la gastroenteritis "
            "(excluye/2). Su severidad base (alto) sube a crítico por ser "
            "cachorro."
        ),
        especie="perro", edad=5, sexo="macho",
        hallazgos=["vomito", "diarrea_con_sangre", "anorexia"],
        esperado_diagnosticos=["parvovirosis"],
        esperado_prioridad="critico",
        no_incluye=["gastroenteritis_aguda"],
        severidades={"parvovirosis": "critico"},
    ),
    CasoPrueba(
        numero="7",
        categoria="Complementario",
        titulo="Exclusión entre diagnósticos (gato macho)",
        objetivo=(
            "El caso cumple el perfil de cistitis, pero la retención urinaria "
            "en un gato macho concluye obstrucción uretral, que la excluye "
            "mediante \\+ excluido/2."
        ),
        especie="gato", edad=36, sexo="macho",
        hallazgos=["esfuerzo_para_orinar", "ausencia_de_orina",
                   "sangre_en_orina", "vocalizacion_dolor"],
        esperado_diagnosticos=["obstruccion_uretral"],
        esperado_prioridad="critico",
        no_incluye=["cistitis"],
    ),
    CasoPrueba(
        numero="8",
        categoria="Complementario",
        titulo="Mismo gato sin retención urinaria",
        objetivo=(
            "Sin 'ausencia_de_orina' no hay retención, la obstrucción no es "
            "candidata y la cistitis deja de estar excluida."
        ),
        especie="gato", edad=36, sexo="macho",
        hallazgos=["esfuerzo_para_orinar", "miccion_frecuente",
                   "sangre_en_orina", "vocalizacion_dolor"],
        esperado_diagnosticos=["cistitis"],
        esperado_prioridad="medio",
        no_incluye=["obstruccion_uretral"],
    ),
    CasoPrueba(
        numero="9",
        categoria="Complementario",
        titulo="Signo de alarma sin diagnóstico",
        objetivo=(
            "Las convulsiones no completan ningún perfil, pero son un signo de "
            "alarma: la primera cláusula de prioridad_caso/3 (con corte) fija "
            "la prioridad en crítico y indicacion_para/3 cambia la indicación "
            "por defecto por la de urgencias."
        ),
        especie="perro", edad=24, sexo="macho",
        hallazgos=["convulsiones"],
        esperado_diagnosticos=["sin_diagnostico_concluyente"],
        esperado_prioridad="critico",
        esperado_alarmas=["convulsiones"],
```

```
        indicacion_contiene={"sin_diagnostico_concluyente": "urgencias"},
    ),
    CasoPrueba(
        numero="10",
        categoria="Complementario",
        titulo="Datos del paciente no válidos",
        objetivo=(
            "Especie y edad fuera del dominio: datos_paciente_validos/1 falla y "
            "la cláusula por defecto de analizar_caso/10 responde "
            "datos_invalidos sin error en Python."
        ),
        especie="conejo", edad="abc", sexo="macho",
        hallazgos=["vomito"],
        esperado_diagnosticos=[],
        esperado_prioridad="datos_invalidos",
    ),
]


class Tee:
    """Escribe simultáneamente en la consola y en un archivo."""

    def __init__(self, *destinos):
        self.destinos = destinos

    def write(self, texto):
        for d in self.destinos:
            d.write(texto)

    def flush(self):
        for d in self.destinos:
            d.flush()


def comprobar(caso: CasoPrueba, r: ResultadoCaso, todos: list[str] | None) -> list[str]:
    """Devuelve la lista de discrepancias entre lo esperado y lo obtenido."""
    fallos = []
    if sorted(r.ids) != sorted(caso.esperado_diagnosticos):
        fallos.append(f"diagnósticos {sorted(r.ids)} !=
{sorted(caso.esperado_diagnosticos)}")
    if r.prioridad != caso.esperado_prioridad:
        fallos.append(f"prioridad {r.prioridad} != {caso.esperado_prioridad}")
    if len(r.ids) != len(set(r.ids)):
        fallos.append("la lista de diagnósticos contiene duplicados")
    if caso.primero and (not r.ids or r.ids[0] != caso.primero):
        fallos.append(f"el primer diagnóstico debería ser {caso.primero}")
    for excluido in caso.no_incluye:
        if excluido in r.ids:
            fallos.append(f"{excluido} debería estar excluido")
    if caso.esperado_ignorados is not None and r.ignorados != caso.esperado_ignorados:
        fallos.append(f"ignorados {r.ignorados} != {caso.esperado_ignorados}")
    if caso.esperado_alarmas is not None and r.alarmas != caso.esperado_alarmas:
        fallos.append(f"alarmas {r.alarmas} != {caso.esperado_alarmas}")
    severidad_obtenida = {d.id: d.severidad for d in r.diagnosticos}
    for diag, nivel in caso.severidades.items():
        if severidad_obtenida.get(diag) != nivel:
            fallos.append(f"severidad de {diag}: {severidad_obtenida.get(diag)} != {nivel}")
    indicaciones = {d.id: d.indicacion for d in r.diagnosticos}
    for diag, texto in caso.indicacion_contiene.items():
        if texto not in indicaciones.get(diag, ""):
            fallos.append(f"la indicación de {diag} debería mencionar «{texto}»")
    if todos is not None and todos != sorted(set(caso.esperado_diagnosticos)):
        fallos.append(f"diagnosticar_todos/5 devolvió {todos}")
    return fallos


def describir_esperado(caso: CasoPrueba) -> list[str]:
    diags = ", ".join(caso.esperado_diagnosticos) or "(ninguno)"
    lineas = [f"diagnósticos = [{diags}]",
              f"prioridad = {ETIQUETA_NIVEL.get(caso.esperado_prioridad,
caso.esperado_prioridad)}"]
```

```
277     if caso.primero:
278         lineas.append(f"primero en la lista = {caso.primero}")
279     if caso.severidades:
280         lineas.append("severidades: " + ", ".join(f"{d}={n}" for d, n in
    caso.severidades.items()))
281     if caso.no_incluye:
282         lineas.append("no debe incluir: " + ", ".join(caso.no_incluye))
283     if caso.esperado_ignorados is not None:
284         lineas.append(f"ignorados = {caso.esperado_ignorados}")
285     if caso.esperado_alarmas is not None:
286         lineas.append(f"alarmas = {caso.esperado_alarmas}")
287     for diag, texto in caso.indicacion_contiene.items():
288         lineas.append(f"la indicación de {diag} menciona «{texto}»")
289     return lineas
290
291
292 def ejecutar(caso: CasoPrueba, motor: MotorTriaVet) -> bool:
293     print("—" * ANCHO)
294     print(f"PRUEBA {caso.numero} · {caso.categoria}: {caso.titulo}")
295     print("—" * ANCHO)
296     print(f"Objetivo : {caso.objetivo}")
297     print(f"Entrada  : especie={caso.especie}, edad={caso.edad} meses, sexo={caso.sexo}")
298     print(f"           hallazgos={caso.hallazgos}")
299
300     resultado = motor.analizar_caso(caso.especie, caso.edad, caso.sexo, caso.hallazgos)
301     print(f"Consulta : ?- {resultado.consulta}.")
302
303     todos = None
304     if caso.verificar_diagnosticar_todos:
305         todos = motor.diagnosticar_todos(caso.especie, caso.edad, caso.sexo, caso.hallazgos)
306         consulta = motor.consulta_diagnosticar_todos(caso.especie, caso.edad, caso.sexo,
307                                                      caso.hallazgos)
308         print(f"           ?- {consulta}.")
309
310     esperado = describir_esperado(caso)
311     print(f"Esperado : {esperado[0]}")
312     for linea in esperado[1:]:
313         print(f"           {linea}")
314
315     print(f"Obtenido : diagnósticos = [{', '.join(resultado.ids) or '(ninguno)'}]"
316           f" · prioridad = {ETIQUETA_NIVEL.get(resultado.prioridad, resultado.prioridad)}")
317     if todos is not None:
318         print(f"           diagnosticar_todos/5 -> {todos}")
319     print(formatear_resultado(resultado, sangria="           "))
320
321     fallos = comprobar(caso, resultado, todos)
322     if fallos:
323         print("Veredicto: ✗ FALLÓ")
324         for f in fallos:
325             print(f"           - {f}")
326     else:
327         print("Veredicto: ✓ PASÓ")
328     print()
329     return not fallos
330
331
332 def seleccionar_casos(argumentos: list[str]) -> list[CasoPrueba]:
333     """Sin argumentos: todos los casos. Con números: solo esos casos."""
334     if not argumentos:
335         return CASOS
336     pedidos = set(argumentos)
337     desconocidos = pedidos - {c.numero for c in CASOS}
338     if desconocidos:
339         raise SystemExit(f"Casos inexistentes: {', '.join(sorted(desconocidos))}. "
340                          f"Disponibles: 1 a {len(CASOS)}.")
341     return [c for c in CASOS if c.numero in pedidos]
342
343
344 def main() -> int:
345     if hasattr(sys.stdout, "reconfigure"):
346         sys.stdout.reconfigure(encoding="utf-8")
```

```
347     casos = seleccionar_casos(sys.argv[1:])
348     CARPETA_EVIDENCIA.mkdir(exist_ok=True)
349     nombre = ("salida_pruebas.txt" if casos is CASOS
350               else "prueba_" + "_".join(c.numero for c in casos) + ".txt")
351     ruta_evidencia = CARPETA_EVIDENCIA / nombre
352     resultados: list[tuple[CasoPrueba, bool]] = []
353     aprobadas = 0
354 
355     with open(ruta_evidencia, "w", encoding="utf-8") as archivo:
356         consola = sys.stdout
357         sys.stdout = Tee(consola, archivo)
358         try:
359             print("=" * ANCHO)
360             print(" TriaVet · Script de pruebas por consola (Python + SWI-Prolog vía
    PySwip)")
361             print(f" Ejecutado: {datetime.now():%Y-%m-%d %H:%M:%S}")
362             print("=" * ANCHO)
363             motor = MotorTriaVet()
364             print(f"Base de conocimiento cargada: {motor.ruta_base.name}\n")
365 
366             resultados = [(c, ejecutar(c, motor)) for c in casos]
367 
368             aprobadas = sum(ok for _, ok in resultados)
369             print("=" * ANCHO)
370             print(" RESUMEN")
371             print("=" * ANCHO)
372             for caso, ok in resultados:
373                 marca = "✓" if ok else "✗"
374                 print(f" {marca} Prueba {caso.numero:>2} · {caso.categoria}: {caso.titulo}")
375             print(f"\n {aprobadas} de {len(resultados)} pruebas aprobadas.")
376         finally:
377             sys.stdout = consola
378 
379     print(f"Evidencia guardada en: {ruta_evidencia}")
380     return 0 if aprobadas == len(resultados) else 1
381 
382 
383 if __name__ == "__main__":
384     raise SystemExit(main())
```

Apéndice E

Salida Completa de pruebas.py

```
================================================================================
 TriaVet · Script de pruebas por consola (Python + SWI-Prolog vía PySwip)
 Ejecutado: 2026-10-21 11:57:12
================================================================================
Base de conocimiento cargada: base_conocimiento.pl

──────────────────────────────────────────────────────────────────────────────
PRUEBA 1 · Caso normal 1: Diagnóstico único claro
──────────────────────────────────────────────────────────────────────────────
Objetivo : Comprobar el encadenamiento de 3 niveles: los signos activan el estado intermedio abdomen_agudo (Nivel 2), que junto con el perfil lleva a una única conclusión de severidad crítica (Nivel 3).
Entrada  : especie=perro, edad=48 meses, sexo=macho
           hallazgos=['raza_torax_profundo', 'abdomen_distendido', 'arcadas_improductivas', 'salivacion_excesiva', 'inquietud']
Consulta : ?- analizar_caso('perro', 48, 'macho', ['raza_torax_profundo', 'abdomen_distendido', 'arcadas_improductivas', 'salivacion_excesiva', 'inquietud'], Validos, Ignorados, Alarmas, Diagnosticos, Prioridad, Recomendacion).
Esperado : diagnósticos = [dilatacion_torsion_gastrica]
           prioridad = CRÍTICO
           severidades: dilatacion_torsion_gastrica=critico
Obtenido : diagnósticos = [dilatacion_torsion_gastrica] · prioridad = CRÍTICO
           Prioridad del caso : CRÍTICO
           Recomendación      : URGENCIA VITAL: acuda de inmediato a un servicio veterinario de urgencias.
           Hallazgos válidos  : abdomen distendido, arcadas improductivas, inquietud, raza torax profundo, salivacion excesiva
             1. Dilatación-torsión gástrica  [CRÍTICO]  certeza 95 %
                Sustento   : abdomen distendido, arcadas improductivas, salivacion excesiva, inquietud, raza torax profundo
                Indicación : No le dé agua ni comida ni intente provocarle el vómito. Trasládelo de inmediato a urgencias.
Veredicto: ✓ PASÓ

──────────────────────────────────────────────────────────────────────────────
PRUEBA 2 · Caso normal 2: Múltiples diagnósticos y recolección
──────────────────────────────────────────────────────────────────────────────
Objetivo : Comprobar que setof/3 reúne varias conclusiones en una sola lista sin duplicados (la entrada repite 'vomito'), que cada una tiene su propia severidad y que el resultado se ordena de mayor a menor urgencia. También verifica diagnosticar_todos/5.
Entrada  : especie=perro, edad=30 meses, sexo=hembra
           hallazgos=['vacunacion_completa', 'ingesta_xilitol', 'vomito', 'vomito', 'temblores', 'poliuria', 'polidipsia', 'perdida_de_peso']
Consulta : ?- analizar_caso('perro', 30, 'hembra', ['vacunacion_completa', 'ingesta_xilitol', 'vomito', 'vomito', 'temblores', 'poliuria', 'polidipsia', 'perdida_de_peso'], Validos, Ignorados, Alarmas, Diagnosticos, Prioridad, Recomendacion).
           ?- diagnosticar_todos('perro', 30, 'hembra', ['vacunacion_completa', 'ingesta_xilitol', 'vomito', 'vomito', 'temblores', 'poliuria', 'polidipsia', 'perdida_de_peso'], Diagnosticos).
Esperado : diagnósticos = [enfermedad_renal_cronica, intoxicacion]
           prioridad = CRÍTICO
           primero en la lista = intoxicacion
           severidades: intoxicacion=critico, enfermedad_renal_cronica=medio
           no debe incluir: gastroenteritis_aguda
Obtenido : diagnósticos = [intoxicacion, enfermedad_renal_cronica] · prioridad = CRÍTICO
           diagnosticar_todos/5 -> ['enfermedad_renal_cronica', 'intoxicacion']
           Prioridad del caso : CRÍTICO
           Recomendación      : URGENCIA VITAL: acuda de inmediato a un servicio veterinario de urgencias.
           Hallazgos válidos  : ingesta xilitol, perdida de peso, polidipsia, poliuria, temblores, vacunacion completa, vomito
             1. Intoxicación  [CRÍTICO]  certeza 85 %
                Sustento   : ingesta xilitol, vomito, temblores
                Agravantes : compromiso neurologico
```

```
47                 Indicación : No induzca el vómito ni dé remedios caseros sin indicación
   veterinaria. Lleve el empaque o una muestra de la sustancia.
48              2. Enfermedad renal crónica  [MEDIO]  certeza 75 %
49                 Sustento   : poliuria, polidipsia, perdida de peso, vomito
50                 Indicación : Mantenga agua fresca disponible siempre y programe consulta
   para pruebas de función renal.
51 Veredicto: ✓ PASÓ
52
53 ═══════════════════════════════════════════════════════════════════════════════
54 PRUEBA 3 · Caso de exclusión: Negación por fallo (\+) y corte (!)
55 ═══════════════════════════════════════════════════════════════════════════════
56 Objetivo : Un cachorro con diarrea con sangre sería candidato a parvovirosis, pero la
   vacunación completa la descarta con \+. Queda la gastroenteritis aguda, cuya severidad base
   (bajo) sube a medio por ser cachorro: el corte en etapa_vida/3 y en severidad/3 elige una
   sola rama.
57 Entrada  : especie=perro, edad=5 meses, sexo=macho
58            hallazgos=['vacunacion_completa', 'vomito', 'diarrea_con_sangre', 'anorexia']
59 Consulta : ?- analizar_caso('perro', 5, 'macho', ['vacunacion_completa', 'vomito',
   'diarrea_con_sangre', 'anorexia'], Validos, Ignorados, Alarmas, Diagnosticos, Prioridad,
   Recomendacion).
60 Esperado : diagnósticos = [gastroenteritis_aguda]
61            prioridad = MEDIO
62            severidades: gastroenteritis_aguda=medio
63            no debe incluir: parvovirosis
64 Obtenido : diagnósticos = [gastroenteritis_aguda] · prioridad = MEDIO
65            Prioridad del caso : MEDIO
66            Recomendación      : PRIORITARIO: programe una consulta veterinaria en las
   próximas 24 a 48 horas y vigile la evolución.
67            Hallazgos válidos  : anorexia, diarrea con sangre, vacunacion completa, vomito
68              1. Gastroenteritis aguda inespecífica  [MEDIO]  certeza 70 %
69                 Sustento   : vomito, diarrea con sangre, anorexia
70                 Agravantes : paciente vulnerable
71                 Indicación : Ofrezca agua en pequeñas cantidades y vigile; consulte si los
   signos duran más de 24 horas o empeoran.
72 Veredicto: ✓ PASÓ
73
74 ═══════════════════════════════════════════════════════════════════════════════
75 PRUEBA 4 · Caso borde 1: Datos insuficientes / lista parcial
76 ═══════════════════════════════════════════════════════════════════════════════
77 Objetivo : La lista cumple solo parte del perfil del moquillo (falta 'fiebre', requisito
   obligatorio verificado con contiene_todos/2) e incluye un hallazgo inexistente que
   particionar/3 debe ignorar. setof/3 no encuentra soluciones y se aplica el valor por
   defecto.
78 Entrada  : especie=perro, edad=60 meses, sexo=hembra
79            hallazgos=['secrecion_nasal', 'tos', 'vacunacion_incompleta', 'estornudo_raro']
80 Consulta : ?- analizar_caso('perro', 60, 'hembra', ['secrecion_nasal', 'tos',
   'vacunacion_incompleta', 'estornudo_raro'], Validos, Ignorados, Alarmas, Diagnosticos,
   Prioridad, Recomendacion).
81 Esperado : diagnósticos = [sin_diagnostico_concluyente]
82            prioridad = BAJO
83            no debe incluir: moquillo
84            ignorados = ['estornudo_raro']
85 Obtenido : diagnósticos = [sin_diagnostico_concluyente] · prioridad = BAJO
86            Prioridad del caso : BAJO
87            Recomendación      : NO URGENTE: vigile al paciente en casa y consulte si los
   signos persisten o empeoran.
88            Hallazgos válidos  : secrecion nasal, tos, vacunacion incompleta
89            Hallazgos ignorados: estornudo_raro
90              1. Sin diagnóstico concluyente  [BAJO]  certeza 0 %
91                 Indicación : Los hallazgos no permiten orientar un diagnóstico; observe la
   evolución y consulte si aparecen nuevos signos.
92 Veredicto: ✓ PASÓ
93
94 ═══════════════════════════════════════════════════════════════════════════════
95 PRUEBA 5 · Caso borde 2: Lista vacía sin errores en Python
96 ═══════════════════════════════════════════════════════════════════════════════
97 Objetivo : Con una lista vacía, Prolog debe responder con la cláusula por defecto (prioridad
   indeterminado y ningún diagnóstico) y Python debe mostrarlo sin lanzar excepciones.
98 Entrada  : especie=gato, edad=24 meses, sexo=hembra
99            hallazgos=[]
```

```
Consulta : ?- analizar_caso('gato', 24, 'hembra', [], Validos, Ignorados, Alarmas, Diagnosticos, Prioridad, Recomendacion).
Esperado : diagnósticos = [(ninguno)]
           prioridad = INDETERMINADO
           alarmas = []
Obtenido : diagnósticos = [(ninguno)] · prioridad = INDETERMINADO
           Prioridad del caso : INDETERMINADO
           Recomendación      : SIN HALLAZGOS VÁLIDOS: no es posible orientar el caso; revise los signos ingresados.
           Hallazgos válidos  : (ninguno)
           Diagnósticos       : (ninguno)
Veredicto: ✓ PASÓ

──────────────────────────────────────────────────────────────────────────────
PRUEBA 6 · Complementario: Contraste del caso 3 sin vacunación
──────────────────────────────────────────────────────────────────────────────
Objetivo : Con los mismos signos del caso 3 pero sin vacunación completa, la parvovirosis sí se concluye y excluye a la gastroenteritis (excluye/2). Su severidad base (alto) sube a crítico por ser cachorro.
Entrada  : especie=perro, edad=5 meses, sexo=macho
           hallazgos=['vomito', 'diarrea_con_sangre', 'anorexia']
Consulta : ?- analizar_caso('perro', 5, 'macho', ['vomito', 'diarrea_con_sangre', 'anorexia'], Validos, Ignorados, Alarmas, Diagnosticos, Prioridad, Recomendacion).
Esperado : diagnósticos = [parvovirosis]
           prioridad = CRÍTICO
           severidades: parvovirosis=critico
           no debe incluir: gastroenteritis_aguda
Obtenido : diagnósticos = [parvovirosis] · prioridad = CRÍTICO
           Prioridad del caso : CRÍTICO
           Recomendación      : URGENCIA VITAL: acuda de inmediato a un servicio veterinario de urgencias.
           Hallazgos válidos  : anorexia, diarrea con sangre, vomito
             1. Parvovirosis canina  [CRÍTICO]  certeza 80 %
                Sustento   : diarrea con sangre, vomito, anorexia
                Agravantes : paciente vulnerable
                Indicación : Aísle al cachorro de otros perros, no lo obligue a comer y llévelo hoy mismo al veterinario: la deshidratación avanza rápido.
Veredicto: ✓ PASÓ

──────────────────────────────────────────────────────────────────────────────
PRUEBA 7 · Complementario: Exclusión entre diagnósticos (gato macho)
──────────────────────────────────────────────────────────────────────────────
Objetivo : El caso cumple el perfil de cistitis, pero la retención urinaria en un gato macho concluye obstrucción uretral, que la excluye mediante \+ excluido/2.
Entrada  : especie=gato, edad=36 meses, sexo=macho
           hallazgos=['esfuerzo_para_orinar', 'ausencia_de_orina', 'sangre_en_orina', 'vocalizacion_dolor']
Consulta : ?- analizar_caso('gato', 36, 'macho', ['esfuerzo_para_orinar', 'ausencia_de_orina', 'sangre_en_orina', 'vocalizacion_dolor'], Validos, Ignorados, Alarmas, Diagnosticos, Prioridad, Recomendacion).
Esperado : diagnósticos = [obstruccion_uretral]
           prioridad = CRÍTICO
           no debe incluir: cistitis
Obtenido : diagnósticos = [obstruccion_uretral] · prioridad = CRÍTICO
           Prioridad del caso : CRÍTICO
           Recomendación      : URGENCIA VITAL: acuda de inmediato a un servicio veterinario de urgencias.
           Hallazgos válidos  : ausencia de orina, esfuerzo para orinar, sangre en orina, vocalizacion dolor
             1. Obstrucción uretral felina  [CRÍTICO]  certeza 85 %
                Sustento   : esfuerzo para orinar, ausencia de orina, vocalizacion dolor
                Indicación : No espere a que orine por sí solo ni le presione el abdomen. Trasládelo de inmediato a urgencias.
Veredicto: ✓ PASÓ

──────────────────────────────────────────────────────────────────────────────
PRUEBA 8 · Complementario: Mismo gato sin retención urinaria
──────────────────────────────────────────────────────────────────────────────
Objetivo : Sin 'ausencia_de_orina' no hay retención, la obstrucción no es candidata y la cistitis deja de estar excluida.
```

```
Entrada  : especie=gato, edad=36 meses, sexo=macho
           hallazgos=['esfuerzo_para_orinar', 'miccion_frecuente', 'sangre_en_orina', 'vocalizacion_dolor']
Consulta : ?- analizar_caso('gato', 36, 'macho', ['esfuerzo_para_orinar', 'miccion_frecuente', 'sangre_en_orina', 'vocalizacion_dolor'], Validos, Ignorados, Alarmas, Diagnosticos, Prioridad, Recomendacion).
Esperado : diagnósticos = [cistitis]
           prioridad = MEDIO
           no debe incluir: obstruccion_uretral
Obtenido : diagnósticos = [cistitis] · prioridad = MEDIO
           Prioridad del caso : MEDIO
           Recomendación      : PRIORITARIO: programe una consulta veterinaria en las próximas 24 a 48 horas y vigile la evolución.
           Hallazgos válidos  : esfuerzo para orinar, miccion frecuente, sangre en orina, vocalizacion dolor
             1. Cistitis / infección urinaria baja  [MEDIO]  certeza 95 %
                Sustento   : miccion frecuente, sangre en orina, esfuerzo para orinar, vocalizacion dolor
                Indicación : Asegure agua fresca y arenero o paseos frecuentes; programe consulta. Si deja de orinar, acuda a urgencias.
Veredicto: ✓ PASÓ

──────────────────────────────────────────────────────────────────────────────
PRUEBA 9 · Complementario: Signo de alarma sin diagnóstico
──────────────────────────────────────────────────────────────────────────────
Objetivo : Las convulsiones no completan ningún perfil, pero son un signo de alarma: la primera cláusula de prioridad_caso/3 (con corte) fija la prioridad en crítico y indicacion_para/3 cambia la indicación por defecto por la de urgencias.
Entrada  : especie=perro, edad=24 meses, sexo=macho
           hallazgos=['convulsiones']
Consulta : ?- analizar_caso('perro', 24, 'macho', ['convulsiones'], Validos, Ignorados, Alarmas, Diagnosticos, Prioridad, Recomendacion).
Esperado : diagnósticos = [sin_diagnostico_concluyente]
           prioridad = CRÍTICO
           alarmas = ['convulsiones']
           la indicación de sin_diagnostico_concluyente menciona «urgencias»
Obtenido : diagnósticos = [sin_diagnostico_concluyente] · prioridad = CRÍTICO
           Prioridad del caso : CRÍTICO
           Recomendación      : URGENCIA VITAL: acuda de inmediato a un servicio veterinario de urgencias.
           Hallazgos válidos  : convulsiones
           Signos de alarma   : convulsiones
             1. Sin diagnóstico concluyente  [CRÍTICO]  certeza 0 %
                Agravantes : compromiso neurologico
                Indicación : Hay signos de alarma aunque no se identifica la causa: acuda de inmediato a urgencias veterinarias.
Veredicto: ✓ PASÓ

──────────────────────────────────────────────────────────────────────────────
PRUEBA 10 · Complementario: Datos del paciente no válidos
──────────────────────────────────────────────────────────────────────────────
Objetivo : Especie y edad fuera del dominio: datos_paciente_validos/1 falla y la cláusula por defecto de analizar_caso/10 responde datos_invalidos sin error en Python.
Entrada  : especie=conejo, edad=abc meses, sexo=macho
           hallazgos=['vomito']
Consulta : ?- analizar_caso('conejo', 'abc', 'macho', ['vomito'], Validos, Ignorados, Alarmas, Diagnosticos, Prioridad, Recomendacion).
Esperado : diagnósticos = [(ninguno)]
           prioridad = DATOS NO VÁLIDOS
Obtenido : diagnósticos = [(ninguno)] · prioridad = DATOS NO VÁLIDOS
           Prioridad del caso : DATOS NO VÁLIDOS
           Recomendación      : DATOS DEL PACIENTE NO VÁLIDOS: revise la especie (perro/gato), la edad en meses (0-300) y el sexo (macho/hembra).
           Hallazgos válidos  : (ninguno)
           Hallazgos ignorados: vomito
           Diagnósticos       : (ninguno)
Veredicto: ✓ PASÓ

==============================================================================
 RESUMEN
```

```
210 ================================================================================
211  ✓ Prueba  1 · Caso normal 1: Diagnóstico único claro
212  ✓ Prueba  2 · Caso normal 2: Múltiples diagnósticos y recolección
213  ✓ Prueba  3 · Caso de exclusión: Negación por fallo (\+) y corte (!)
214  ✓ Prueba  4 · Caso borde 1: Datos insuficientes / lista parcial
215  ✓ Prueba  5 · Caso borde 2: Lista vacía sin errores en Python
216  ✓ Prueba  6 · Complementario: Contraste del caso 3 sin vacunación
217  ✓ Prueba  7 · Complementario: Exclusión entre diagnósticos (gato macho)
218  ✓ Prueba  8 · Complementario: Mismo gato sin retención urinaria
219  ✓ Prueba  9 · Complementario: Signo de alarma sin diagnóstico
220  ✓ Prueba 10 · Complementario: Datos del paciente no válidos
221
222  10 de 10 pruebas aprobadas.
```